

INCLUSIVE WEB PLATFORM

By:

**Zaidan Mansour
Charbel Rizk**

Supervisor(s):

**Dr. Jean Yves Youssef
Dr. Roger Obeid
Dr. Rima Kilany**

A report submitted in partial fulfillment of the requirements for the degree of
COMPUTER AND COMMUNICATIONS ENGINEERING

September 2026



Ecole Supérieure d'Ingénieurs de Beyrouth
Université Saint-Joseph de Beyrouth

1 Abstract

Persons with disabilities continue to face digital and organizational barriers in recruitment, even when employment initiatives are designed to promote inclusion. This project extends the JoIn Hospitality Phase I prototype into a multilingual, accessible, and human-controlled platform for hospitality employment and training. The implemented system combines a React/Vite interface, a Symfony security and business-workflow boundary, PostgreSQL persistence, a deterministic Python compatibility service, and a separate voice/profile-assistance service. Candidates can use manual entry or editable speech transcription in English, French, or Arabic; explicit consent is required before profile text is processed, and no AI suggestion is saved without candidate confirmation. Employers publish controlled hospitality positions or training programs, configure practical and educational requirements, identify important catalogue tasks, and record available assistance. Matching remains deterministic and explainable rather than using an opaque learned model.

Verification combines unit, integration, browser, internationalization, accessibility, performance, deployment, and human-evaluation evidence. The current evidence includes 17/17 deterministic-scoring tests, 52/52 voice/action-safety tests, 25/25 API integration checks, 26/26 browser end-to-end checks, a passing production build, a three-locale/seven-namespace translation contract, and 104 automated WCAG route-scenario scans with no recorded axe findings, overflow, console errors, or failed requests. A free cloud staging environment was deployed through Render and Supabase. Its 8 September snapshot exposed 15 active job descriptions, 923 catalogue tasks, 14,453 disability-task assessments, and 74 published opportunities. An initial on-site accessibility session at Arc en Ciel included a real candidate and a blind JAWS user; its protocol and detailed results remain to be completed in this draft. The report therefore presents strong prototype evidence while avoiding unsupported claims of full WCAG conformance, production availability, or real-world matching effectiveness.

Keywords: accessible recruitment; persons with disabilities; hospitality; WCAG 2.1; screen reader; voice navigation; human-in-the-loop AI; deterministic matching; multilingual interface; cloud staging.

Contents

1	Abstract	2
	List of figures	7
	List of tables	7
	Acronyms.....	8
2	Introduction.....	9
3	Background Research and Related work.....	11
3.1	Historical and Conceptual Background.....	11
3.2	Inclusive Recruitment and Existing Platforms	11
3.3	Phase I Limitations and Need for Dataset Expansion	12
3.4	Accessibility and Multimodal Digital Interfaces	13
3.5	Speech Recognition and NLP for Candidate Profile Creation	13
3.6	Human-in-the-Loop AI and Ethical Considerations	15
3.7	Critical Evaluation of Existing Approaches	15
3.8	Contribution of the Proposed Work	16
4	Requirements	17
4.1	Functional Requirements	17
4.2	Non-Functional Requirements: Encompassing measurable Quality Attributes, Constraints, Standards, and Regulations.....	21
4.3	Conceptual model.....	27
5	Solution Design.....	32
5.1	Propose multiple solutions: Design alternatives and comparison	32
	Alternative 1: Fully Monolithic Web Application	33
	Alternative 2: Full Microservices Architecture.....	34
	Alternative 3: Layered Modular Architecture with External Service Integration— Chosen Approach	34
	A. Authentication and Role Management.....	36
	B. PwD Candidate Verification.....	37
	D. AI-Assisted NLP Workflow	37
	E. Matching Approach	38
5.2	Decision	42
5.3	Alternatives evaluated	42
5.4	Selected boundary.....	42
5.5	Reason and remaining trade-off.....	42
5.6	Authentication.....	42

5.7	Server session; JWT API token; external identity provider	42
5.8	Symfony JWT plus server-confirmed role session	42
5.9	Fits the SPA/API baseline and explicit RBAC; staging now uses HTTPS and secret-managed JWT keys, while expiry/revocation governance and production monitoring remain required.	42
5.10	Candidate verification	42
5.11	Self-declaration; automated OCR/registry; authorized human review	42
5.12	Human review with private temporary evidence	42
5.13	Avoids unsupported automated eligibility decisions; creates reviewer workload and governance duties.....	42
5.14	Candidate input	42
5.15	Manual only; voice only; multimodal.....	42
5.16	Manual form plus optional editable text/voice profile assistant	42
5.17	Maximizes access and graceful degradation; multilingual accuracy requires formal evaluation.	42
5.18	AI extraction	42
5.19	Rules; automatic LLM persistence; reviewable LLM suggestions.....	42
5.20	Consent-gated suggestions with human confirmation.....	42
5.21	Handles natural language while preventing unreviewed sensitive data from becoming fact. 42	
5.22	Matching.....	42
5.23	Keywords; opaque learned decision; deterministic task scoring.....	42
5.24	Versioned deterministic weighted task policy.....	42
5.25	Explainable and testable; future calibration needs expert-reviewed examples rather than cosmetic score changes.	42
5.26	Private files.....	43
5.27	Public upload; database blob; private filesystem/object store	43
5.28	Private Symfony storage locally and Supabase object storage in staging	43
5.29	Enforces authenticated access and deletion; staging uses purpose-specific Supabase buckets, while production encryption, backup, and erasure governance remain required.	43
5.30	Notifications.....	43
5.31	Email only; in-app only; both plus outcome log.....	43
5.32	Plain-text email and accessible in-app notice with minimized event log	43
5.33	Business decisions survive mail failure; production deliverability monitoring remains required.	43
5.34	Caching.....	43
5.35	No cache; process-local memory; shared Redis/equivalent.....	43
5.36	Measurement-driven shared cache planned.....	43

5.37	Add only with TTL, invalidation, privacy exclusions, observability, and database fallback.....	43
5.38	Accessibility verification	43
5.39	Automated only; manual only; combined evidence	43
5.40	Automated scans/tests plus keyboard, zoom, reflow, and screen-reader plan.....	43
5.41	No AA claim until the remaining rendered/manual evidence is completed.....	43
5.42		43
5.43	Solution modeling	43
6	Implementation and Evaluation	46
6.1	Implemented System	46
6.2	Engineering Alternatives and Decision Rationale	46
6.3	AI-Assisted Profile-Building Workflow.....	48
6.4	Accessibility and Responsive Engineering	49
6.5	Security, Verification, and Privacy	50
6.6	Catalogue and Scoring Policy	51
6.7	Integration, E2E, and Performance Evidence	52
6.8	Evaluation Strategy.....	53
6.9	Cloud Staging Deployment and Operational Validation	54
6.10	Human-Centred Accessibility Evaluation	55
6.11	Requirements Traceability	56
6.12	Current Limitations	58
7	Conclusion, Limitations, and Future Work	58
7.1	Remaining Validation and Future Work.....	58
7.2	Risk Management.....	59
7.3	Project Management.....	59
8	References	60
9	Appendix A — System Architecture, Setup, and Operations	61
9.1	Implemented Runtime Architecture	61
9.2	Prerequisites and First-Time Setup	62
9.3	Daily Operation, Validation, and Troubleshooting	62
10	Appendix B — Authentication, Candidate Verification, Security, and Privacy	63
10.1	Authentication and Authorization Model	63
10.2	Candidate Eligibility Verification	63
10.3	Private Storage, Consent, and Candidate Rights.....	63
10.4	Security Verification and Deployment Duties.....	65
11	Appendix C — AI-Assisted Profile Building and Voice Navigation	65
11.1	Candidate Profile Assistant.....	65

11.2	Voice-Navigation Safety Architecture	65
11.3	Language, Privacy, Configuration, and Limitations	66
12	Appendix D — Deterministic Scoring Engine and Catalogue Policy	66
12.1	Authoritative Data Flow	66
12.2	Mathematical Model	66
12.3	Eligibility, Ranking, and Interpretation	67
12.4	Catalogue Baseline and Missing Assessments	68
12.5	Determinism, Testing, and Future Calibration	68
13	Appendix E — Responsive Design, Keyboard Access, and WCAG Evidence	68
13.1	Audit Scope and Implemented Corrections	68
13.2	Keyboard Acceptance Plan	70
13.3	ISO 9241-110 Interaction Principles	70
13.4	Conformance Boundary	71
14	Appendix F — Outcome Logging, Accessible Notifications, Testing, and Performance	71
14.1	Outcome and Notification Data	71
14.2	Recorded Automated Verification	71
14.3	Performance Measurements	72
14.4	Required Manual and Deployment Retests	72
15	Appendix G — Documentation and Evidence Traceability	72
16	Appendix H — Cloud Staging Deployment Evidence	75
16.1	Deployment Topology and Boundaries	75
16.2	Security and Data Controls	75
16.3	Data Migration and Verification	75
16.4	Operational Lessons	75
17	Appendix I — Human Evaluation and Statistical Workbook	75
17.1	Participant and Session Record	75
17.2	Task-Level Dataset	75
17.3	Planned Calculations	76
17.4	Evidence Integrity	76

List of figures

Figure 1: C4 notation legend	27	
Figure 2: System context diagram (C4 Level 1)	28	
Figure 3: Container diagram (C4 Level 2)	29	
Figure 4: Conceptual class diagram	31	
Figure 5: Matching-evaluation state diagram	31	
Figure 6: Layered modular runtime service boundaries and health evidence		36
Figure 7: Candidate disability-card requirement and verification consent	37	
Figure 8: Employer task-importance and assistance configuration	39	
Figure 9: Candidate data-access, consent, retention, and erasure controls		40
Figure 10: Component diagram	45	
Figure 11: Candidate AI-assisted profile-building workflow	49	
Figure 12: Keyboard/voice navigation and accessible notifications		50
Figure 13: Authorized candidate-verification dashboard	51	
Figure 14: Responsive role dashboards	52	
Figure 15: Integration, E2E, and performance evidence	53	
Figure 16: Gantt chart	60	

List of tables

Table 1: High-level architecture alternatives comparison	34	
Table 2: Low-level component alternatives and selected boundaries		41
Table 3: Engineering design-decision matrix	47	
Table 4: Requirements progress and verification matrix	56	
Table 5: Runtime services and local endpoints	61	
Table 6: Security and privacy control summary	64	
Table 7: Deterministic scoring policy	66	
Table 8: Accessibility verification coverage	69	
Table 9: Recorded automated verification results	71	
Table 10: Repository documentation incorporated into this report	73	

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
C4	Context, Containers, Components, and Code
CSS	Cascading Style Sheets
CV	Curriculum Vitae
DB	Database
ERD	Entity Relationship Diagram
FYP	Final Year Project
GDPR	General Data Protection Regulation
GUI	Graphical User Interface
HoReCa	Hotels, Restaurants, and Catering
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
NFR	Non-Functional Requirement
PwD	Person with Disability / Persons with Disabilities
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines

2 Introduction

Access to employment is not only an economic need, but also a major factor of dignity, autonomy, social participation, and personal identity. For persons with physical disabilities (PwD), however, access to the labor market remains difficult, especially when recruitment processes are still based on traditional profiles, subjective judgment, and a focus on limitations rather than abilities. In many cases, employers hesitate to hire PwD because disability is perceived as a source of complexity, cost, adaptation constraints, or uncertainty. On the other side, many PwD are not fully informed about their real professional potential, especially in fields such as hospitality and food service, where tasks are diverse and can often be adapted or reorganized.

The JoIn Hospitality initiative addresses this challenge by presenting inclusion not as an act of charity, but as a win-win opportunity for both the hospitality industry and persons with disabilities. Its objective is to break the ice between recruiters and PwD by transforming physical conditions into job-relevant remaining abilities, and by matching these abilities with real professional tasks in the HoReCa sector: hotels, restaurants, catering, and food service. The JoIn Hospitality pitch presents the software component as an “icebreaker” tool that facilitates the connection between recruiters and PwD, while the tailored framework matches professional tasks with remaining functional abilities in order to propose roles that genuinely fit each person’s capabilities.

Phase I of the Inclusive Web Platform delivered a functional proof of concept for this vision. The previous team developed a web-based recruitment platform using a Symfony backend, PostgreSQL database, React frontend, JWT authentication, role-based access control, and a separate Flask microservice for task-level ability matching. The prototype allowed candidates to define their remaining abilities, browse job opportunities, apply to suitable positions, and receive compatibility results based on the relationship between their abilities and the tasks required by each job. The Phase I report clearly identified the central idea of the platform: recruitment should begin with what a person can do, not with what the person cannot do.

However, while Phase I proved the technical feasibility of ability-to-task matching, the accessibility dimension of the platform remains a major challenge. Since the platform is designed for PwD, accessibility cannot be treated as a secondary feature. A recruitment system intended for persons with physical disabilities must be usable by people who may face motor difficulties, visual barriers, speech-related limitations, fatigue, or literacy-related difficulties. In its current form, the application process still depends mainly on reading, typing, and navigating a standard web interface. This creates a paradox: the platform promotes inclusion, but some of the users who most need it may still face barriers when trying to use it.

Phase II therefore focuses on transforming the existing platform into a genuinely accessible and multimodal recruitment experience. The objective is to extend the Inclusive Web Platform by allowing candidates to complete their account creation, profile building, ability declaration, and job application through different input and output channels, including text, voice, audio support, and accessible visual interfaces. In particular, the system will integrate an automatic speech recognition layer capable of transcribing spoken input in the target languages: Lebanese Arabic, French, and English. This transcription will then be processed by an NLP layer that proposes a structured profile of remaining abilities and skills mapped to the platform’s predefined vocabulary.

A key principle of this phase is that artificial intelligence will assist the user, not replace the user’s decision. Any ability, skill, or disability-related attribute proposed by the AI layer must be shown to

the candidate for review, correction, confirmation, or rejection before it is stored or used in the matching engine. This human-in-the-loop approach is essential because disability and health-related information are sensitive, personal, and should never be inferred as fact without explicit confirmation. The confirmed structured profile will then feed the existing Phase I matching engine, allowing the project to improve accessibility and data entry without redesigning the whole matching logic.

In parallel, the user interface will be redesigned and evaluated according to WCAG 2.1 Level AA accessibility principles. This includes keyboard navigation, screen-reader compatibility, sufficient contrast, clear focus management, resizable text, captions or transcripts for audio and video content, and compatibility results that are not communicated by color alone. These improvements are necessary to make the platform accessible not only in its mission, but also in its actual daily use.

The scope of this project is therefore the continuation and engineering extension of the existing Inclusive Web Platform. It does not aim to replace medical assessment, rehabilitation professionals, recruiters, job coaches, or workplace adaptation policies. Instead, it aims to provide a technical bridge between PwD and recruiters by making the application process more accessible, by helping candidates express their abilities through voice or text, and by preserving a transparent ability-to-task matching process. The main beneficiaries are candidates with physical disabilities, recruiters and employers in the hospitality and food service sector, inclusion organizations, and future researchers who may use consent-based outcome data to improve the matching model.

This project constitutes a major design experience because it combines web engineering, software architecture, accessibility engineering, human-computer interaction, applied AI, multilingual speech recognition, NLP-based information extraction, security, privacy, and data-protection design. The expected deliverable is an improved platform prototype that extends the Phase I system with multimodal candidate input, AI-assisted profile extraction, human confirmation, accessibility-compliant interfaces, and integration with the existing task-level matching service. The project will also include updated documentation, functional and non-functional requirements, design diagrams, technical tests, accessibility checks, and, where feasible, feedback from representative users or stakeholders in the inclusion ecosystem.

Another expected contribution of Phase II is the preparation of a foundation for future data-driven improvement. With explicit user consent, the platform may record application outcomes and, when available, placement results. This would allow future teams to build a labelled dataset based on real-world use rather than only expert-defined assumptions. Such a dataset could later support the improvement of the matching model and provide evidence for how well the platform supports inclusive recruitment in practice.

The main challenges expected in this phase are the support of Lebanese Arabic speech recognition, the risk of incorrect AI extraction, the sensitivity of disability-related data, and the difficulty of meeting accessibility requirements in a realistic and measurable way. These challenges will be addressed through modular service design, model evaluation, editable transcriptions, confidence-based review prompts, explicit user confirmation before storing sensitive information, and accessibility validation using automated tools and assistive-technology testing.

This work is carried out in continuity with the previous FYP team's prototype and in cooperation with the project supervisors and JoIn Hospitality stakeholders. It also builds on the broader vision of JoIn Hospitality, which combines software, adaptive work tools, and training to support inclusion in the hospitality industry. Through these improvements, Phase II aims to move the platform from a functional prototype toward a truly inclusive digital recruitment tool: one that does not only make

opportunities visible, but also makes the application process usable, understandable, and empowering for the people it is meant to serve.

Final implementation update (8 September 2026): Phase II has delivered the principal candidate, employer, verifier, and administrator workflows; English/French/Arabic interfaces; candidate-controlled AI profile assistance; bounded voice navigation; deterministic explainable matching; hospitality work and training preferences; controlled employer requirements; private document storage; cloud staging deployment; and repeatable automated evidence. The remaining work is evaluation-oriented: complete and document representative-user sessions, repeat tests across additional devices and networks, analyse the collected usability data, resolve any observed defects, and finalize the report and presentation. A cache remains intentionally deferred because measured local performance is within budget and no deployed bottleneck has yet justified its complexity.

3 Background Research and Related work

3.1 Historical and Conceptual Background

The employment of persons with physical disabilities has evolved significantly over time. In the past, disability was mainly understood through a medical model, where the person was evaluated according to impairment, incapacity, or limitation. This view often led to dependency, protection, and separation from ordinary professional environments. Employment opportunities for persons with disabilities were therefore frequently limited or approached through charity-based and assistance-based models.

Over time, this perspective shifted toward a social and rights-based model. In this model, disability is not seen only as an individual medical condition, but also as the result of barriers created by society, workplaces, recruitment methods, tools, and communication systems. The JoIn Hospitality research material highlights this evolution by comparing the medical model and the social model of disability, and by showing that disability should also be understood through the way society and work environments are organized.

This shift is essential for the present project. If the barriers faced by persons with disabilities are not only physical or medical, but also organizational, digital, and communicational, then software can play an important role in reducing these barriers. A recruitment platform can either reproduce exclusion by forcing all users to interact in the same traditional way, or it can support inclusion by adapting the application process to the user's abilities.

The hospitality and food service sector, also known as HoReCa, represents an important field for this approach. It includes hotels, restaurants, catering, food preparation, service, and related activities. This sector contains a wide variety of tasks and roles, which makes it suitable for task-level analysis. Many jobs in this field can be broken down into smaller tasks, and some tasks may be adapted, reorganized, or partially performed depending on a candidate's remaining abilities. JoIn Hospitality presents the hospitality industry as a strong opportunity for inclusion because it faces labor shortages, high turnover, and a growing need for inclusive recruitment solutions.

3.2 Inclusive Recruitment and Existing Platforms

Most current recruitment platforms are based on traditional candidate profiles, CVs, qualifications, experience, and employer interpretation. Even when platforms aim to support inclusion, many of

them mainly increase the visibility of candidates with disabilities without changing the structure of recruitment itself. In these systems, the employer still evaluates the candidate globally and subjectively, without necessarily understanding whether the candidate can perform the specific tasks required by the job.

Phase I of the Inclusive Web Platform addressed this limitation by introducing an ability-to-task matching approach. Instead of treating a job as a single role, the platform breaks it down into tasks and subtasks, then compares those requirements with the candidate's remaining functional abilities. The Phase I report identified this as the central idea of the platform: recruitment should begin with what a person can do, not with what the person cannot do.

In Lebanon, ProAble was studied as an existing inclusive recruitment platform. It plays an important role by connecting persons with disabilities, NGOs, rehabilitation centers, and employers. However, as explained in the Phase I report, its approach remains mainly profile-based and does not provide task-level matching between job requirements and remaining abilities. This shows that visibility alone is not enough. A candidate may be visible to employers, but still not properly understood if the platform does not explain what tasks the candidate can actually perform.

Human-centered inclusion models, such as the work observed through Arc-en-Ciel and Join Hospitality, provide another important reference. These approaches are grounded in real human abilities, practical observation, coaching, and adaptation. Their strength is that they respect the complexity of disability and avoid reducing the person to a simple profile or score. However, they are difficult to scale digitally if they depend only on manual evaluation, human experts, or case-by-case judgment.

Automated recruitment and AI-based matching systems provide another direction. They can improve speed, scalability, and consistency, but they also introduce risks. If the decision logic is not transparent, automated systems can become difficult to trust. They may also reinforce bias or make incorrect assumptions about a person's abilities. For this reason, the present project does not use AI as a final decision-maker. Instead, AI is used as an assistive layer that helps structure information while keeping the candidate and recruiter in control.

3.3 Phase I Limitations and Need for Dataset Expansion

Phase I successfully demonstrated that task-level ability matching is technically possible. The previous prototype included a React frontend, a Symfony backend, a PostgreSQL database, JWT authentication, role-based access control, and a Flask microservice for the AI matching component. The AI microservice received candidate disability or ability information and returned compatibility results for the available job roles.

However, one important limitation of Phase I was the small size of the job-description dataset. The matching engine was trained and tested using only three HoReCa job roles: Glacerie, Boulangerie-Pâtisserie, and Chocolaterie. The Phase I report states that the training dataset was derived from Excel files containing these three job roles, with task-by-task feasibility ratings per disability type. This was sufficient to prove the concept, but it is not enough to represent the real diversity of the hospitality and food service sector.

In reality, HoReCa includes many different job families, tasks, work environments, physical constraints, and required abilities. A platform limited to three jobs cannot provide enough opportunities for candidates or enough flexibility for recruiters. It also limits the usefulness of the machine learning model, because the model can only predict compatibility for a very small set of roles.

Phase II completed the planned catalogue expansion by importing twelve additional hospitality job descriptions, increasing coverage from three to fifteen active roles. The new datasets use the same ordered-task, disability-assessment, source-reference, and validation model as the original three. This expansion improves practical coverage while remaining subject to final domain-stakeholder review of the added descriptions, translations, and assessments.

Phase II did not retrain or deploy a machine-learning matching model. The project selected deterministic weighted rules because no sufficiently large, representative, consented outcome dataset was available for responsible supervised learning. Catalogue expansion therefore increases the opportunities evaluated by the same explainable policy; a learned model remains a future research option only after outcome data, bias analysis, versioning, and stakeholder approval are available.

3.4 Accessibility and Multimodal Digital Interfaces

Another important limitation of many digital recruitment systems is that they assume all users can read, type, click, scroll, and navigate a web interface in the same way. This assumption is problematic for persons with physical disabilities, who may experience motor difficulties, visual barriers, fatigue, limited hand use, or difficulty using a keyboard and mouse. Therefore, accessibility is not only a design improvement; it is a basic condition for the platform to achieve its purpose.

The Phase I platform promoted inclusive recruitment, but the interaction process still depended mainly on standard web navigation, reading, and typing. This creates a gap between the mission of the platform and the actual user experience. A platform designed for PwD must be accessible not only in its objective, but also in its interface and interaction methods.

The state of the art in accessible web design is strongly influenced by the Web Content Accessibility Guidelines, especially WCAG 2.1. These guidelines are organized around four major principles: content should be perceivable, operable, understandable, and robust. For this project, WCAG 2.1 Level AA is used as the target accessibility level. This includes keyboard navigation, screen-reader support, sufficient color contrast, visible focus indicators, resizable text, alternative text for meaningful visuals, captions or transcripts for audio and video content, and compatibility results that are not communicated by color alone.

Multimodal interaction is also important for this project. A multimodal interface allows users to interact with the system through more than one channel, such as text, speech, audio feedback, visual elements, keyboard navigation, or assistive technologies. In the context of the Inclusive Web Platform, this means that candidates should not be forced to complete their profile or application only by typing. They should be able to use voice input, text input, or a combination of both depending on their abilities and preferences.

3.5 Speech Recognition and NLP for Candidate Profile Creation

Automatic Speech Recognition, or ASR, is the technology that converts spoken language into written text. In this project, ASR is relevant because it allows candidates to describe their abilities, experience, limitations, preferences, and needs orally. This can reduce the dependency on typing and make the application process more accessible for users with upper-limb motor difficulties, fatigue, or low digital literacy.

However, ASR introduces technical challenges. French and English are widely supported by many speech-to-text systems, but Lebanese Arabic is more difficult because it is a dialect with variations in pronunciation, vocabulary, and code-switching. Many Lebanese users naturally mix Arabic,

French, and English in the same sentence. This makes transcription more complex and may reduce accuracy. Therefore, the transcript must not be treated as final. The candidate must be able to review, correct, and confirm the transcription before it is used.

Several ASR solutions may be considered for this project, including open-source or API-based speech-to-text models such as Whisper. These models are relevant because they support multilingual transcription and can reduce the typing effort required from candidates. However, their accuracy must be evaluated in the project context, especially for Lebanese Arabic, mixed-language speech, background noise, and informal expressions. For this reason, the ASR output will always remain editable and subject to candidate confirmation before being processed further.

Natural Language Processing, or NLP, is the second important technology in this phase. Once the candidate's speech or typed text is available, NLP can be used to extract structured information from a free narrative. For example, if a candidate says that they can stand for short periods, use one hand with precision, understand instructions, or prefer seated tasks, the NLP layer can propose corresponding abilities and skills from the platform's predefined vocabulary.

In this project, the NLP component will not rely mainly on a traditional keyword-based or rule-based NLP pipeline. Traditional NLP approaches often depend on predefined keywords, manual rules, or classical text classification techniques. While these methods can be useful in controlled situations, they are less flexible when users express themselves in natural, informal, multilingual, or dialectal language. Since candidates may describe their abilities using Lebanese Arabic, French, English, or a mixture of these languages, a more flexible approach is required.

For this reason, Phase II will use an LLM-assisted NLP approach through the OpenAI API. The objective is to transform the candidate's free narrative, obtained either from typed text or from ASR transcription, into a structured representation of abilities, skills, preferences, and possible constraints. This approach is suitable for the project because OpenAI's API supports structured outputs, where model responses can be constrained to follow a supplied JSON schema, making it easier to integrate extracted information with backend services.

In addition, the LLM-assisted layer can be made context-aware by providing it with the current page or form context. This means that the system does not ask the model to extract every possible piece of information, but only the information relevant to the current step of the user journey, such as account creation, profile completion, ability declaration, or job application. This improves precision and reduces irrelevant extraction.

The extracted information will be mapped to the platform's predefined vocabulary and returned in a structured format, such as JSON, so that it can be processed by the backend and later used by the matching engine. The extracted data can be organized into categories such as functional abilities, skills, limitations or constraints, preferences, assistance needs, and experience-related information, depending on the current form being completed. For example, during the ability declaration step, the system may focus on what the candidate can do, what they can do with assistance, and what should be avoided. During the job application step, it may focus more on preferences, availability, and specific needs related to the selected role. This makes the AI layer more controlled, more understandable, and better aligned with the platform workflow.

The advantage of this approach is that it reduces the burden on the candidate. Instead of manually filling many fields, the candidate can describe their situation naturally, and the system helps transform this narrative into structured data. This makes the application process more accessible and more user-friendly, especially for candidates who may have difficulty typing or navigating long forms.

However, LLM-assisted NLP also has limitations. The model may misunderstand the user, propose an incorrect ability, miss an important detail, or infer sensitive information that the candidate did not intend to declare. Structured output improves integration and format reliability, but it does not guarantee that every extracted value is semantically correct. OpenAI also notes that structured outputs do not prevent all model mistakes inside the generated values.

For this reason, the AI output must not be considered final. The system will use the LLM as an assistive tool, not as an automatic authority. Every proposed ability, skill, preference, or constraint must be displayed to the candidate for review, correction, confirmation, or rejection before it is stored or used by the matching engine.

This human-in-the-loop mechanism is essential for both ethical and technical reasons. It protects the candidate's autonomy, reduces the risk of incorrect profile construction, and prevents sensitive disability-related information from being stored without explicit confirmation. In this way, the ASR and NLP layers support accessibility and efficiency while keeping the candidate in control of their own data.

3.6 Human-in-the-Loop AI and Ethical Considerations

Human-in-the-loop AI is a design approach where artificial intelligence supports the user but does not make final decisions without human review. This principle is essential in the present project because disability-related information is sensitive and personal. The system must not automatically decide that a candidate has a certain limitation, skill, disability attribute, or ability based only on an AI interpretation.

In the proposed Phase II system, the AI layer will propose a structured profile, but the candidate will remain responsible for confirming, editing, or rejecting each suggestion. Nothing sensitive should be stored or used by the matching engine before explicit user confirmation. This reduces the risk of harmful errors and keeps the candidate in control of their own profile.

This approach also supports explainability and trust. Instead of giving a hidden AI result, the system shows which abilities or skills were extracted and allows the user to understand and modify them. This is important because inclusive recruitment should not become another opaque scoring process. The goal is to make the system transparent, understandable, and respectful of the user's autonomy.

Privacy is also a major concern. Disability and health-related information are sensitive data. Therefore, the system should apply data minimization, explicit consent, role-based access control, secure storage, and clear separation between temporary AI suggestions and confirmed user information. The platform should collect only what is necessary for job matching and should avoid storing inferred disability-related information without the candidate's approval.

3.7 Critical Evaluation of Existing Approaches

The related work can be grouped into four main categories: traditional recruitment platforms, inclusive employment platforms, human-centered inclusion models, and AI-assisted digital systems.

Traditional recruitment platforms are scalable and familiar to employers, but they are not designed around disability inclusion. They usually focus on CVs, qualifications, and experience, which can disadvantage PwD if the recruitment process ignores remaining abilities or task adaptability.

Inclusive employment platforms improve visibility and create bridges between candidates, NGOs, and employers. Their main strength is that they focus on a population that is often excluded from ordinary recruitment channels. However, many of them remain profile-based and do not provide a structured ability-to-task matching mechanism.

Human-centered inclusion models are more respectful of individual complexity. They can consider real abilities, workplace context, adaptation, coaching, and human judgment. Their limitation is scalability. A fully manual process is difficult to apply to many candidates, many employers, and many job roles.

AI-assisted systems can process information quickly and support scalable matching. They can help extract skills, classify profiles, or recommend job opportunities. However, they may lack transparency, make incorrect inferences, or introduce bias if not carefully controlled. In the context of disability, this risk is especially serious because incorrect assumptions may affect dignity, opportunity, and trust.

The Phase II solution combines these strengths while making implementation status explicit. It preserves deterministic task-level scoring, implements a controlled and traceable catalogue-import process, expands the active catalogue from three to fifteen job descriptions, adds accessible multimodal interaction, uses AI only as an assistive profile-building layer, and requires human confirmation before suggested profile data is stored.

3.8 Contribution of the Proposed Work

The contribution of Phase II is not to create a completely new recruitment platform, but to extend the existing Inclusive Web Platform into a more complete, accessible, and realistic system. Phase I proved that task-level matching can support more objective inclusive recruitment. Phase II builds on this foundation by improving both the coverage of the matching engine and the accessibility of the candidate journey.

The first data contribution is a controlled catalogue-import pipeline and a documented missing-assessment policy. The active staging catalogue contains fifteen hospitality datasets, 923 normalized tasks, 16 disability categories, and 14,453 recorded assessments. The twelve added datasets use the same schema and traceability rules as the original three; final domain-stakeholder validation remains required and is distinguished from technical import completion.

The second contribution is the preservation and verification of the established mathematical scoring approach. The engine performs deterministic, explainable task-level compatibility calculations using confirmed candidate data, task weights, highlighted or mandatory status, feasibility assessments, education requirements, and assistance availability. AI-generated profile suggestions do not silently change its inputs.

The third contribution is the integration of a multimodal application process. Candidates will be able to use speech, text, or both to build their profile and apply for jobs. By combining ASR, NLP, and human-in-the-loop confirmation, the platform will help candidates express their abilities more naturally while keeping control over the final structured profile.

The fourth contribution is the alignment of the platform with accessibility standards, especially WCAG 2.1 Level AA. This transforms accessibility from a general intention into measurable interface requirements that can be tested and improved.

Finally, Phase II prepares the foundation for future data-driven improvement. With explicit user consent, the platform can record application outcomes and placement results, creating a labelled dataset for future improvement of the matching model. This means that the system can gradually evolve from expert-defined assumptions toward evidence-based matching supported by real-world results.

In summary, this phase contributes by connecting four dimensions that are often separated: inclusive employment, task-level job matching, accessible digital interaction, and AI-assisted profile structuring. The result is a platform that does not only help recruiters see candidates, but also helps candidates express themselves, confirm their data, and access opportunities through an interface adapted to their abilities.

4 Requirements

The Phase II requirements extend the Phase I Inclusive Web Platform. The current implementation covers role-based accounts, candidate eligibility verification, profiles, jobs, applications, task-level ability matching, accessible notifications, outcome logging, private document handling, privacy controls, responsive access, keyboard and optional voice navigation, and AI-assisted profile building. Dataset expansion, deployed capacity validation, representative-user evaluation, and final WCAG rendered verification remain tracked work.

The requirements in this section are structured according to the principles of requirements engineering described in ISO/IEC/IEEE 29148:2011. Functional requirements describe the observable services and behaviors that the system shall provide, while non-functional requirements define measurable quality attributes, technical constraints, and compliance obligations. These requirements will guide the design, implementation, testing, and validation of the platform.

4.1 Functional Requirements

FR1 — User Account, Authentication, and Role Management

The system shall allow users to register, sign in, and log in as candidates, employers, administrators, or authorized verifiers using email and password authentication. Each user account shall be assigned a unique internal identifier generated by the system. The system shall apply role-based access control so that each user can access only the functionalities allowed by their role.

FR2 — Candidate Profile Management

The system shall allow candidates to create, view, edit, and update their profiles. The profile shall include personal information, education level, previous experience, skills, preferences, confirmed remaining abilities, assistance needs, and relevant constraints.

FR3 — Official Disability Recognition Management

The system shall allow candidates, when applicable and with explicit consent, to provide information related to official disability recognition. This may include disability card status, issuing authority, recognition level, disability range, and verification status.

Candidate registration shall require one disability card or official recognition document, explicit consent to its verification use, and email verification. The document shall be accessible only to an authorized verifier or administrator and shall never be exposed to employers.

The platform shall not calculate or validate the official disability percentage by itself. It shall only store confirmed information provided by the candidate or authorized stakeholders.

FR4 — Ability Selection and Management

The system shall allow candidates to select, view, and update their remaining abilities from a predefined vocabulary. These abilities shall be used by the matching engine to compare the candidate profile with job task requirements.

FR5 — Multimodal Application Flow

The system shall allow candidates to complete the main candidate journey with keyboard and text input. Voice navigation is an optional additional control, and the profile-building workflow accepts either typed text or an editable voice transcript.

The same journey shall remain fully operable using keyboard navigation.

FR6 — Speech Capture and Transcription

The system shall capture spoken input from the candidate and transcribe it into editable text using an ASR service. The ASR component shall target Lebanese Arabic, French, and English.

The candidate shall be able to review, correct, and confirm the transcript before it is processed by the AI extraction layer.

FR7 — LLM-Assisted NLP Profile Extraction

The system shall use an LLM-assisted NLP component, through the OpenAI API, to analyze the candidate's typed narrative or ASR transcript.

The AI component shall propose structured information such as functional abilities, skills, limitations or constraints, preferences, assistance needs, and experience-related information.

The extracted information shall be mapped to the platform's predefined vocabulary and returned in a structured format such as JSON.

FR8 — Page-Aware AI Assistance

The system shall guide the LLM-assisted extraction according to the current page or form context.

The AI extraction layer shall be limited to the candidate profile schema and active controlled catalogues. Separately, voice-navigation commands shall use page-specific action allowlists so that spoken commands cannot execute arbitrary application behavior.

This requirement ensures that the AI component extracts only relevant information and avoids unrelated or unnecessary suggestions.

FR9 — Human-in-the-Loop Confirmation

The system shall present every AI-proposed ability, skill, limitation, preference, assistance need, or disability-related attribute to the candidate for review, correction, confirmation, or rejection.

No sensitive information shall be stored or used in matching before explicit user confirmation.

FR10 — Job, Task, and Subtask Management

The system shall allow administrators and authorized employers to create, view, edit, and manage job roles.

Each job role shall be divided into tasks and subtasks. Each task or subtask shall be associated with required abilities, feasibility conditions, and possible adaptation needs.

FR11 — Job Dataset Expansion

The system shall extend the existing job-description dataset from three HoReCa roles to fifteen roles by importing twelve additional structured descriptions through the controlled validation pipeline. Status: implemented technically; domain-stakeholder validation remains required.

Each new job description shall be decomposed into tasks and subtasks and mapped to the platform's predefined ability vocabulary.

FR12 — Task Feasibility Classification

The system shall classify task feasibility using three main categories:

- feasible;
- feasible with adaptation or assistance;
- not suggested.

The system shall avoid negative or discriminatory wording such as “cannot”. Instead, it shall use professional and inclusive wording such as “not suggested” or “requires adaptation”.

FR13 — Task Weighting Configuration

The system shall support assigning weights to tasks and subtasks according to their importance in the job role.

Task weights may be based on essentiality, frequency, safety impact, hygiene or quality impact, delegability, and possibility of adaptation.

These weights shall be used to calculate more realistic compatibility scores.

FR14 — Ability-Task Compatibility Matching

The system shall compare the candidate's confirmed remaining abilities with the task and subtask requirements of each job role.

The system shall calculate a compatibility score between the candidate and each job role.

FR15 — Scoring Engine Preservation and Validation

The system shall preserve and test the approved deterministic task-level mathematical scoring policy while importing additional validated job catalogues. Any future change to the scoring formula or its inputs shall require a separately documented version, validation, and stakeholder review.

The Python scoring service shall generate explainable compatibility scores across supported jobs and expose task-level feasibility, weights, mandatory-task effects, and assistance assumptions without using disability identity as an employment decision.

FR16 — Partial Compatibility Evaluation

The system shall evaluate compatibility at task level before calculating the overall job score.

For each task, the system shall indicate whether the task is feasible, feasible with adaptation or assistance, or not suggested for the candidate.

FR17 — Accessible Compatibility Result Visualization

The system shall present compatibility results in a clear and accessible way.

The result page shall show the overall compatibility score, task-level feasibility, suggested adaptations where applicable, and explanations that are understandable to candidates and recruiters.

The visualization shall not rely on color alone.

FR18 — Job Browsing and Application Handling

The system shall allow candidates to browse job opportunities, view job details, view task requirements, see compatibility results, and submit applications.

The system shall allow employers to view, review, and manage applications received for their posted jobs.

FR19 — Candidate Search and Filtering

The system shall allow employers to search and filter candidates using job-relevant criteria such as confirmed abilities, skills, location, availability, compatibility score, and adaptation needs.

Filtering based directly on disability identity or unsupported sensitive assumptions shall be avoided unless it is legally justified, consent-based, and approved by the project stakeholders.

FR20 — Application Status Tracking

The system shall allow employers to update the status of an application.

Application statuses may include pending, in review, interview requested, accepted, and rejected.

Candidates shall be able to view the current status of their applications.

FR21 — Accessible Notifications

The system shall notify users of important events, including application updates, interview requests, profile confirmation steps, and matching results.

Notifications shall be available through accessible in-app notifications and email notifications.

FR22 — Accessible Output and Feedback

The system shall provide accessible feedback through clear text messages, screen-reader-compatible status messages, and optional text-to-speech reading of key screens or results.

For any audio or video content used in the platform, the system shall provide captions or transcripts when applicable.

FR23 — Outcome Logging for Future Learning

With explicit user consent, the system shall record application outcomes and, when available, placement outcomes.

These records shall support the creation of a labelled dataset for future improvement of the matching model using real-world data.

4.2 Non-Functional Requirements: Encompassing measurable Quality Attributes, Constraints, Standards, and Regulations**4.2.1 Quality attributes and technical constraints**

Non-functional requirements define the measurable qualities and constraints that determine whether the system is usable, secure, accessible, reliable, and maintainable. Functional requirements may be implemented correctly while the final product remains difficult or unsafe to use if these quality attributes are not respected. Therefore, the following requirements include measurable targets for performance, accessibility, security, privacy, reliability, compatibility, and maintainability.

NFR1 — Accessibility

The platform shall target WCAG 2.1 Level AA conformance.

This includes keyboard navigation, screen-reader compatibility, semantic HTML and ARIA where needed, visible focus indicators, sufficient color contrast, resizable text, alternative text for meaningful visuals, captions or transcripts for media, and result visualization not based on color alone.

Accessibility shall be validated using automated tools such as axe or Lighthouse. The target is to have no critical accessibility violations on the main candidate pages. At least one manual keyboard-only test and one screen-reader test shall be performed where feasible.

NFR2 — Performance

Under normal operating conditions, core pages such as login, dashboard, profile, job search, and application submission shall respond in under 3 seconds for at least 90% of user requests.

The system shall support at least 100 concurrent active users during prototype testing without major degradation of the main workflows.

Compatibility matching for a candidate and a job role shall complete in under 2 seconds for at least 90% of matching requests under normal workload.

AI-assisted profile extraction shall return suggestions within approximately 3 to 5 seconds for short candidate narratives under normal load.

ASR transcription of a short utterance shall return within a few seconds, depending on the selected ASR service and network conditions.

NFR3 — Privacy and Special-Category Data Protection

Disability-related and health-related information, including official disability cards or recognition documents, shall be treated as sensitive personal data.

The system shall collect only necessary information, request explicit consent before storing sensitive data, and avoid storing AI-inferred disability-related attributes without user confirmation.

Uploaded official disability documents shall be used only for verification purposes and shall not be exposed to employers by default.

NFR4 — Security

The system shall build on secure authentication and role-based access control. Passwords shall be stored using secure hashing, and protected API routes shall only be accessible to authorized users according to their role.

Access to uploaded official disability cards or recognition documents shall be restricted to authorized administrators or approved verification stakeholders only.

Communication between the frontend, backend, matching service, ASR service, LLM API, notification service, and storage components shall use secure channels where applicable and shall minimize unnecessary personal identifiers.

NFR5 — Explainability and User Control

The system shall not present AI-proposed abilities, skills, limitations, or disability-related attributes as facts before candidate confirmation.

All AI suggestions shall be transparent, editable, confirmable, and rejectable.

The candidate shall be able to understand what information was extracted and how it will be used by the platform.

NFR6 — Multilingual Support and Graceful Degradation

The system shall target Lebanese Arabic, French, and English for voice and text input.

When transcription or extraction confidence is low, the system shall ask the user to review, correct, repeat, or clarify the input instead of automatically committing uncertain information.

If the ASR or LLM service is unavailable, the user shall still be able to continue the workflow using manual text input.

NFR7 — Reliability and Availability

The system shall preserve confirmed user data in the event of unexpected failure.

Confirmed profile information, uploaded documents, applications, and matching results shall not be lost if an ASR, LLM, or matching service temporarily fails.

The platform shall aim for 99% availability during the prototype deployment period, excluding scheduled maintenance.

NFR8 — Maintainability and Modularity

ASR, LLM-assisted NLP, and deterministic task-level scoring shall remain separable modules or services so that providers and policies can be replaced or versioned without redesigning the whole platform.

This modular design shall allow models, APIs, or services to be replaced, updated, or retrained without redesigning the entire platform.

The project shall preserve the existing microservice-oriented pattern used in Phase I.

NFR9 — Compatibility and Responsive Access

The platform shall be usable on recent versions of modern web browsers.

The platform shall be responsive and usable on desktop and mobile browsers.

A native mobile application is outside the scope of this phase and may be considered future work.

NFR10 — Data Quality and Model Evaluation

Every imported job dataset shall follow the controlled catalogue schema. The current staging baseline contains fifteen active descriptions, 923 tasks, 16 disability categories, and 14,453 recorded task/disability assessments. Imports preserve source references and are rejected when validation fails; technical import does not replace domain-stakeholder approval.

Each job shall be decomposed into tasks and subtasks with feasibility labels and, where applicable, task weights.

The deterministic scoring engine shall be evaluated against expected expert-defined compatibility outputs, policy invariants, catalogue integrity checks, and regression tests. Predictive-model metrics shall be used only if a future, separately approved learned model is introduced.

NFR11 — Notification Delivery

Email and in-app notifications shall be delivered within one minute of the triggering event under normal operating conditions, excluding delays caused by external email providers.

NFR12 — Ethical Use of AI

The system shall use AI to assist profile creation and matching, not to make final employment decisions.

Final recruitment decisions shall remain the responsibility of recruiters and stakeholders.

The platform shall avoid unsupported assumptions, discriminatory filtering, and hidden automated decisions.

NFR13 — Secure Document Handling

Uploaded official disability cards or recognition documents shall be stored securely and accessed only by authorized roles.

The system shall not display uploaded documents to employers by default. Employers may see only a verification status such as not provided, pending, verified, or rejected.

NFR14 — Scalability

The system architecture shall support future expansion beyond fifteen job roles without requiring a complete redesign.

The job, task, ability, and matching structures shall allow additional HoReCa roles and other sectors to be added later.

NFR15 — Usability

The user interface shall be simple, consistent, and understandable for candidates, employers, and administrators.

Candidates shall be able to complete the main application journey without requiring technical knowledge or training.

Error messages, confirmation messages, and AI suggestions shall be written in clear and user-friendly language.

NFR16 — Caching and Response Optimization

The system shall use a caching layer to improve response time and reduce repeated processing for frequently accessed or computationally expensive data.

The cache may store non-sensitive data such as job descriptions, task lists, ability vocabulary, accessibility metadata, and repeated compatibility results when appropriate.

Sensitive information such as official disability documents, raw transcripts, unconfirmed AI outputs, and private candidate data shall not be cached unless protected, justified, and compliant with privacy requirements.

Cached data shall have expiration and invalidation rules to ensure that users receive updated results when profiles, jobs, tasks, abilities, or matching models change.

4.2.2 Standards, laws and regulations

The project must consider standards and regulations related to accessibility, privacy, security, requirements engineering, software validation, usability, and system modeling. These standards ensure that the platform is not only functional, but also safe, accessible, maintainable, and ethically appropriate for a sensitive inclusion-related domain.

Name of standard / regulation	Part / Section	Part / Section title	Brief description of how the solution is compliant
WCAG 2.1	Level AA	Web Content Accessibility Guidelines	The platform targets accessible navigation and interaction through keyboard operability, screen-reader support, sufficient contrast, visible focus, resizable text, alternative text, captions/transcripts, and compatibility results that are not based on color alone.
GDPR	Article 5	Principles relating to processing of personal data	The system applies data minimization, purpose limitation, transparency, and storage limitation. Only necessary candidate, ability, document, and matching data are collected.
GDPR	Article 6	Lawfulness of processing	Candidate data are processed only for explicit recruitment and matching purposes, based on consent or another applicable legal basis defined by the project stakeholders.
GDPR	Article 9	Special categories of personal data	Disability and health-related information are treated as special-category data. AI-inferred information and uploaded official disability documents are not stored or used without explicit confirmation and consent.
GDPR	Article 25	Data protection by design and by default	The system includes privacy safeguards in its design, such as human confirmation, limited data collection, role-based access, and separation between temporary AI suggestions and confirmed stored profile data.
GDPR	Article 32	Security of processing	The platform uses authentication, role-based access control, password hashing, restricted document access, and secure handling of sensitive data.
ISO/IEC/IEEE 29148:2011	Requirements engineering processes	Systems and software engineering — Life cycle processes —	The project separates functional and non-functional requirements, phrases them as observable system behaviors and measurable constraints, and uses

	and products	Requirements engineering	them as a basis for design, implementation, testing, and validation.
ISO/IEC 27001:2013	A.9.1	Access control policy	Role-based access control restricts functionalities and sensitive information to authorized candidates, employers, and administrators. Uploaded official disability documents are not visible to employers by default.
ISO/IEC 27001:2013	A.12.4.1	Event logging	Security-relevant and workflow-relevant actions, such as login, profile updates, job changes, matching requests, document verification, and application status updates, can be logged for audit and troubleshooting.
IEEE 12207	6.4.11	Software validation process	The project includes validation through functional tests, accessibility tests, ASR/NLP tests, matching evaluation, and user feedback where feasible.
IEEE 12207	6.3.5	Software configuration management	The project uses Git/GitHub for version control, change tracking, collaboration, and code management.
ISO 9241	Part 110	Dialogue principles	The interface follows principles of consistency, feedback, error tolerance, user control, and suitability for the task. This supports accessible and understandable candidate journeys.
ISO/IEC 25010	Product quality model	Usability, accessibility, reliability, security, maintainability, compatibility	The solution addresses usability, accessibility, security, reliability, maintainability, compatibility, and performance through measurable non-functional requirements.
UML 2.5	Modeling notation	Unified Modeling Language	UML diagrams may be used to describe system structure, workflows, components, classes, and interactions in a standardized way.
OWASP Web Security Guidelines	Web application security practices	Secure web application development	The platform considers secure authentication, authorization, input validation, protected API routes, restricted document access, and reduced exposure of sensitive data.
Local disability recognition and employment	Country-specific	Official disability recognition and inclusion obligations	The platform may store official disability recognition information when voluntarily provided and confirmed by the candidate. The system does not

inclusion regulations			calculate official disability percentage and does not replace legal or medical recognition processes.
-----------------------	--	--	---

4.3 Conceptual model

This section describes the conceptual model of the proposed Inclusive Web Platform for employment. Based on the gathered functional and non-functional requirements, the following section provides an overview of the primary entities that make the whole system, as well as of its processes and the actors involved. Diagrams are included to demonstrate how the system operates, what concepts the platform relies on, and how the components are related. That is why, being an intermediate step between requirement analysis and solution design outlined further in the report, the conceptual model plays a significant role in describing the proposed system.

3.3.1 Context Diagram (C4 - Level 1)

Before presenting the system context diagram, a short legend is provided to clarify the notation used in the C4 model. The person symbol represents an external actor who interacts with the platform, such as a candidate, employer, or administrator. The blue rectangle represents the software system under study, while the dashed arrow represents a relationship or interaction between an actor and the system.

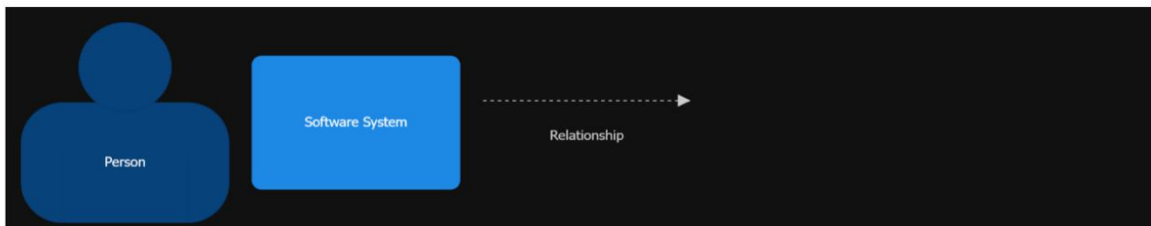


Figure 1: C4 notation legend

The system context diagram gives a high-level view of the Phase II Inclusive Employment Platform, its main users, and its external notification service. It defines the system boundary without showing internal technical components, which are detailed later in the container diagram.

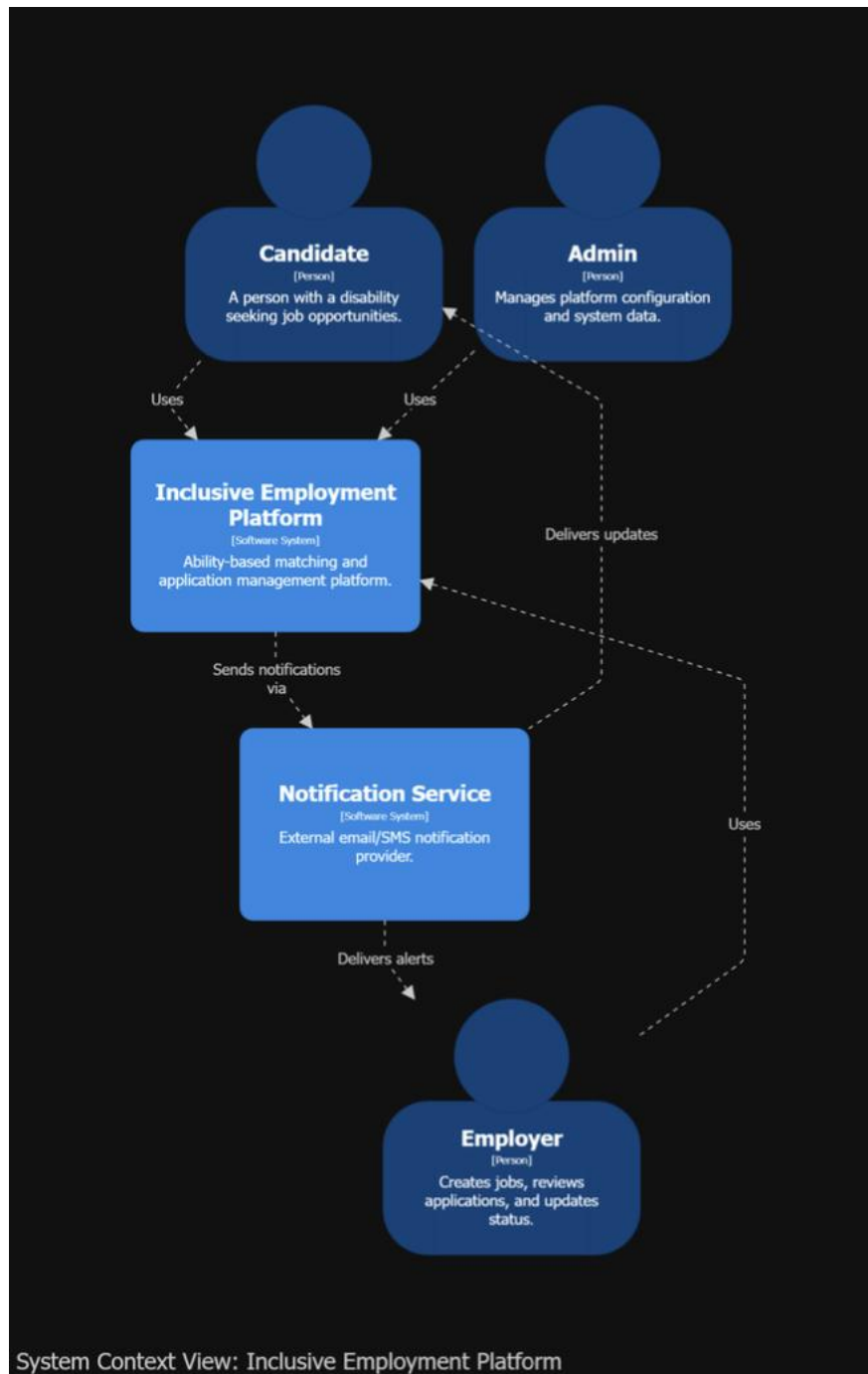


Figure 2: System context diagram (C4 Level 1)

The platform is used by candidates, employers, and administrators. Candidates create profiles, view compatible jobs, and apply. Employers post jobs, review applications, and update application status. Administrators manage platform data and configuration. The notification service delivers alerts and application updates to users.

3.3.2 Container Diagram (C4 - Level 2)

The container diagram shows the main technical blocks and their communication boundaries: the accessible React interface, authoritative Symfony API, PostgreSQL, deterministic Python scoring service, AI/voice service, private file storage, and notification delivery. The cache is a planned performance component and is not represented as already implemented.

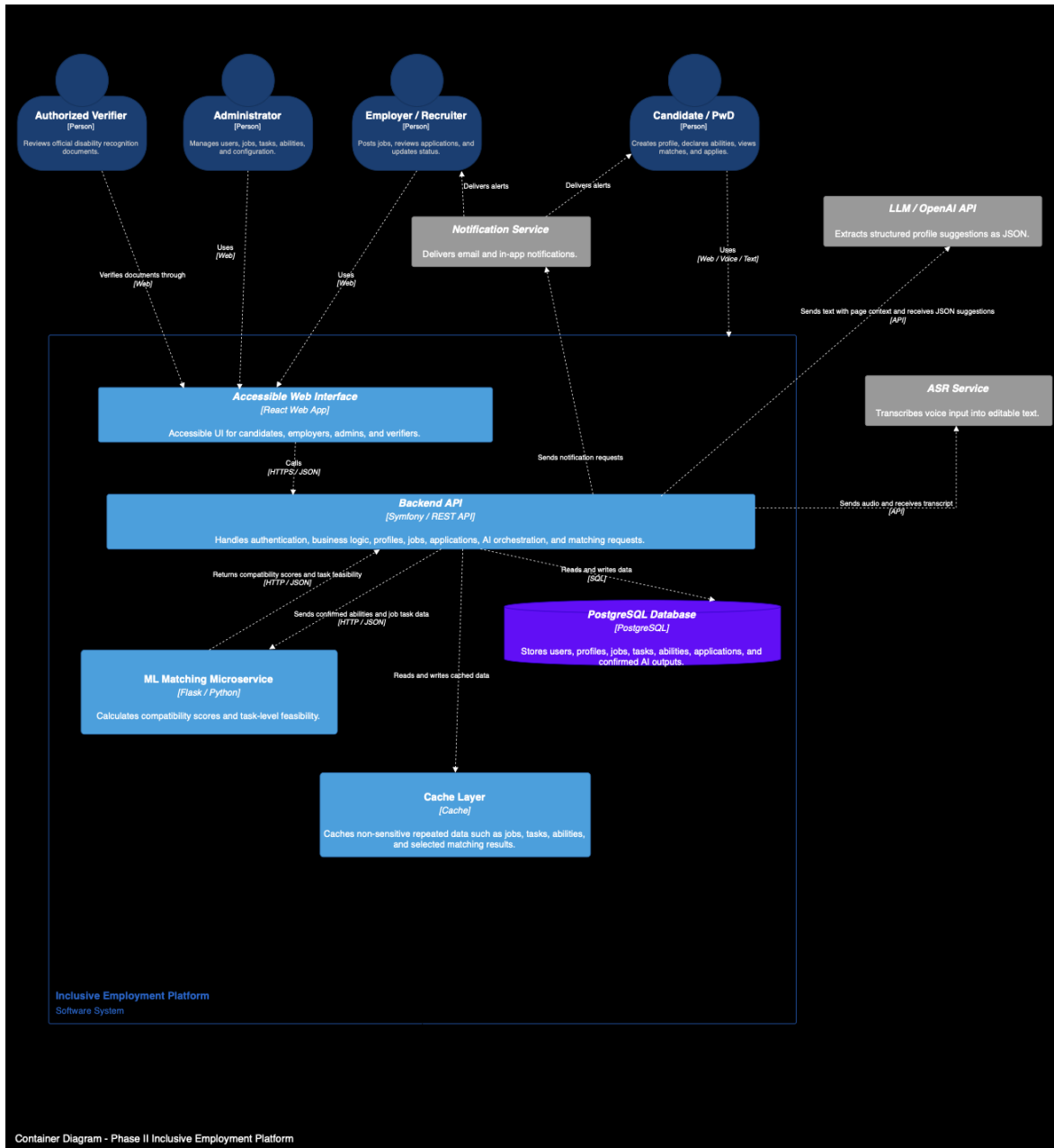


Figure 3: Container diagram (C4 Level 2)

The accessible web interface is used by PwD candidates, employers, administrators, and authorized verifiers. It communicates with the backend API through HTTPS/JSON requests. The

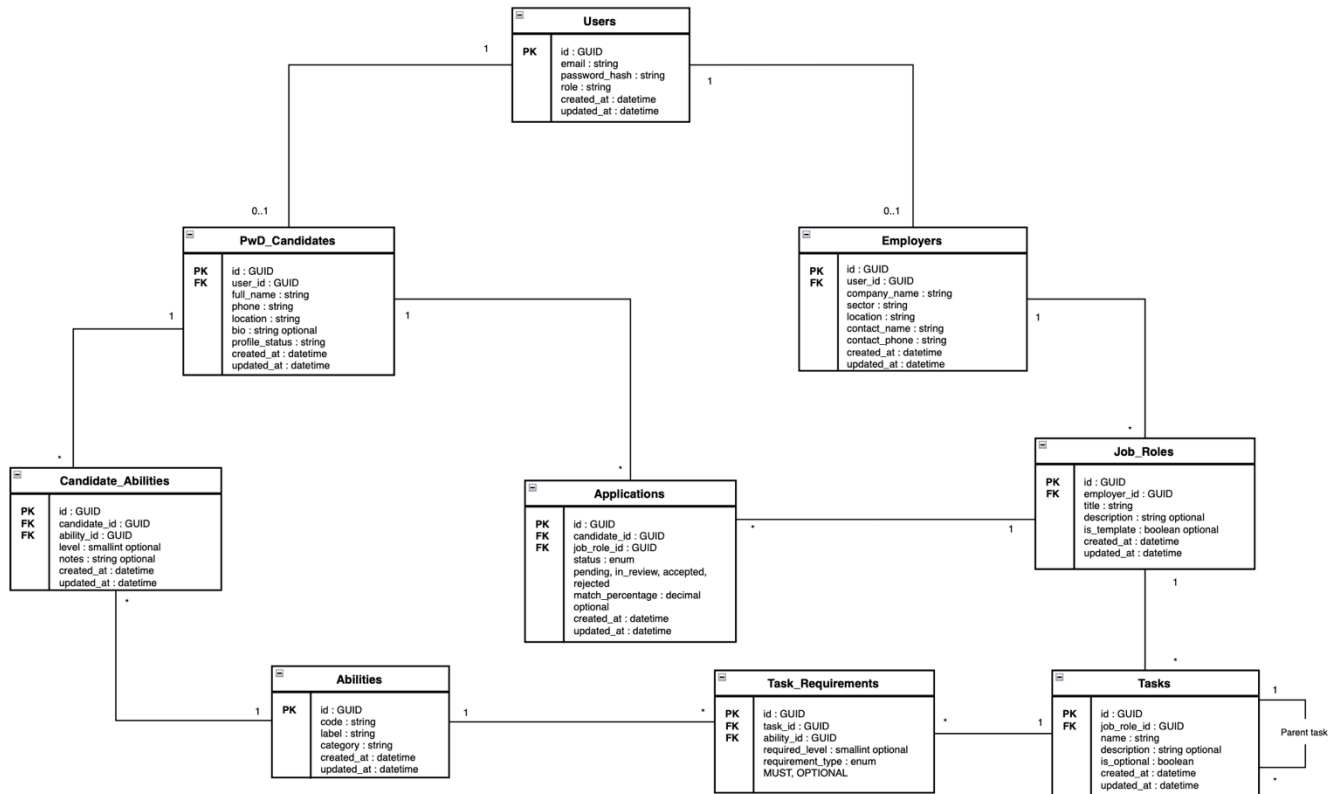
backend API manages authentication, role-based access, profiles, jobs, applications, AI orchestration, matching requests, and verification workflows.

PostgreSQL stores confirmed records such as users, profiles, job definitions, tasks, assessments, applications, verification status, outcomes, consents, and audit metadata. The deterministic Python service receives authoritative candidate and offer data from Symphony and returns compatibility, eligibility, ranking, and task-level explanations. The planned cache will be limited to repeated non-sensitive catalogue data and will require explicit expiry and invalidation rules.

External services support the platform with additional features. The notification service delivers email and in-app alerts, the ASR service transcribes voice input into editable text, and the LLM/OpenAI API extracts structured profile suggestions from text using the current page context. These AI suggestions are not stored as final data before the candidate reviews and confirms them.

3.3.3 Conceptual Class Diagram

The conceptual class diagram presents the main entities used by the platform and the relationships between them. The `Users` entity stores account information and user roles. The `PwD_Candidates` and `Employers` entities store profile information for the two main public user types. PwD candidates can declare their abilities through `Candidate_Abilities`, apply to available `Job_Roles`, and receive a matching percentage for each application. Employers create job roles, and each job role is divided into tasks. Each task is connected to the abilities required to perform it

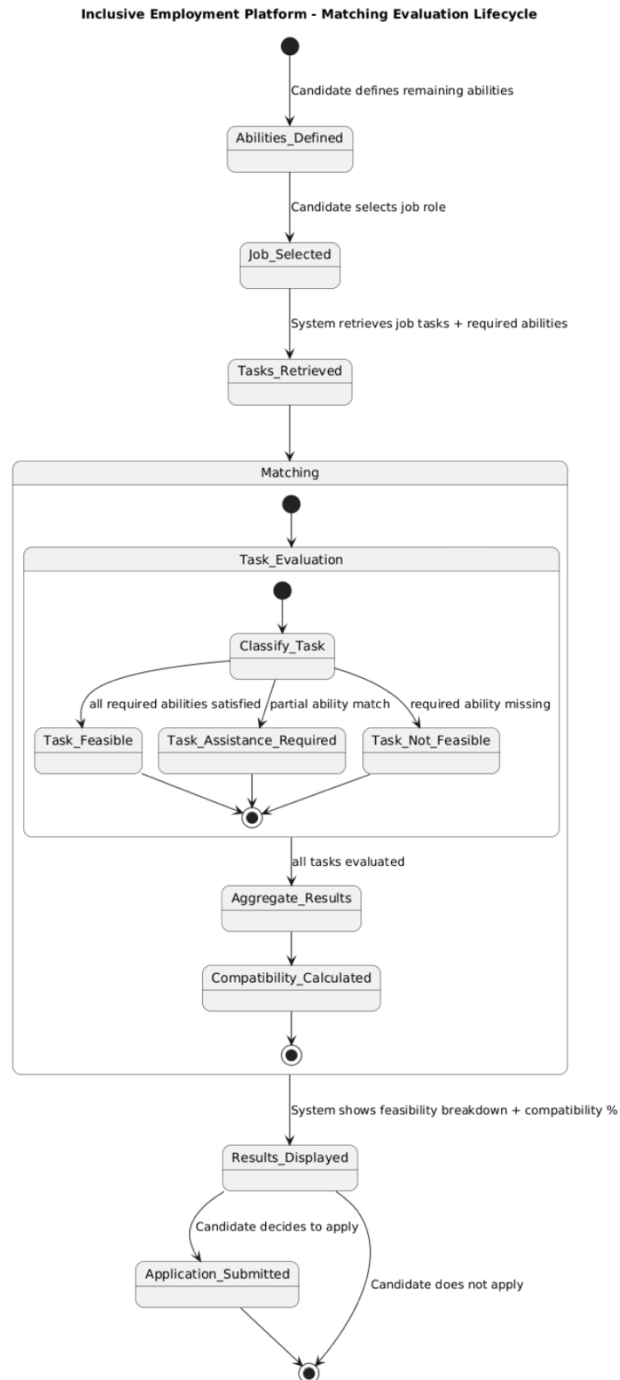


through `Task_Requirements`. This structure supports the platform's ability-based job matching process.

Figure 4: Conceptual class diagram

3.3.4 State Diagram

Figure 5: Matching-evaluation state diagram



The state diagram shows the lifecycle of the job matching evaluation process. After the PwD candidate defines their remaining abilities and selects a job role, the system retrieves the required tasks and abilities for that job. Each task is then evaluated and classified according to the candidate's abilities. After all tasks are evaluated, the system aggregates the results, calculates the compatibility percentage, and displays the feasibility breakdown. Finally, the candidate can either submit an application or decide not to apply.

5 Solution Design

The previous section defined what the system should do through its functional and non-functional requirements, conceptual model, and behavioral diagrams. This section explains how the proposed Phase II Inclusive Employment Platform will satisfy these requirements through concrete design choices, architecture decisions, technologies, and processing workflows.

The solution is designed as an accessible, ability-based recruitment platform for PwD candidates and employers in the hospitality field. It supports candidate profile creation, disability recognition verification, multimodal input, AI-assisted profile extraction, ability-based job matching, application management, and notification delivery. The design follows a modular layered architecture in order to separate the user interface, backend logic, database, matching service, AI services, secure file handling, and external notifications.

The main design objective is to keep the platform simple enough to implement and maintain, while still supporting the new Phase II requirements such as accessibility, speech input, LLM-assisted NLP, human-in-the-loop confirmation, secure handling of sensitive files, and compatibility scoring.

5.1 Propose multiple solutions: Design alternatives and comparison

There is no single solution for designing the Phase II Inclusive Employment Platform. Several design alternatives were considered before selecting the final architecture. The comparison was done at both high level and low level in order to choose a solution that satisfies the functional requirements, respects the non-functional requirements, and remains realistic for implementation.

The platform must support several important features: user registration, role-based access, PwD candidate verification, candidate profile creation, ability declaration, job and task management, application tracking, multimodal input, AI-assisted extraction, matching calculation, notifications, and secure handling of sensitive data. At the same time, the solution must remain accessible, secure, maintainable, scalable, and easy to use for persons with disabilities.

The selected solution is a layered modular web architecture. It combines a React interface, Symfony API, PostgreSQL database, Python scoring service, private file storage, separate AI/voice processing, notifications, and a planned cache layer for repeated non-sensitive data.

This design was selected because it gives a balance between simplicity and modularity. The platform remains easier to implement than a full microservices architecture, but the most

important components, such as the matching engine, ASR service, LLM-assisted extraction, and notification service, remain separated enough to be tested, replaced, or improved later.

A key design objective is to reduce digital barriers without making AI or voice mandatory. The complete manual profile form remains available. In the optional profile workflow, a candidate types or records English, French, or Arabic; recorded speech is transcribed into editable text; explicit consent is collected; and the AI service returns structured, reviewable profile suggestions. The workflow does not automatically populate registration or job-application fields.

However, the system does not save AI-generated information directly. The extracted content is first shown to the candidate as editable suggestions. The candidate must review, modify, confirm, or reject the information before submission. Only confirmed data is stored in the database. This supports the human-in-the-loop principle and prevents the system from storing incorrect or unconfirmed information.

Accessibility is a major design criterion. The interface supports keyboard operation, visible focus, persistent labels, responsive reflow, screen-reader-oriented semantics, error recovery, and status announcements across the principal workflows. These corrections target WCAG 2.1 Level AA, but the project does not claim conformance until the documented post-remediation browser, native 200% zoom, and assistive-technology tests are completed.

The service is designed to be simple and free for PwD candidates, making the platform easier to use and more inclusive. This design choice supports the project objective of helping persons with disabilities access employment opportunities without being blocked by complex digital forms or inaccessible recruitment procedures.

5.1.1 High level alternatives

At the global architecture level, three main alternatives were considered: a fully monolithic application, a full microservices architecture, and a layered modular architecture with external service integration.

Alternative 1: Fully Monolithic Web Application

In this approach, all functionalities would be implemented inside one single application. User management, candidate profiles, employer features, job management, matching logic, AI processing, notifications, and file handling would all be part of the same backend. This solution would be simple to deploy and easier to understand at the beginning. It would also reduce communication overhead between components because everything would run in one application.

However, this approach is not flexible enough for Phase II. The deterministic scoring policy, speech transcription, AI extraction, notification delivery, and private-file lifecycle have different dependencies and change rates. If every concern were embedded in one backend, replacing the transcription provider or versioning the scoring policy would increase regression risk across unrelated workflows.

Therefore, a fully monolithic approach was not selected.

Alternative 2: Full Microservices Architecture

In this approach, the system would be divided into several independent services. For example, there could be a separate authentication service, profile service, job service, matching service, AI service, notification service, and file storage service.

This architecture provides strong modularity and scalability. Each service can be deployed and updated independently. It is useful for large-scale systems with many users and high traffic.

However, this option adds significant complexity. It requires service discovery, API gateways, monitoring, inter-service security, deployment orchestration, and more advanced infrastructure. For an FYP prototype, this complexity is not justified. The expected system size and workload do not require a full microservices architecture at this stage.

Therefore, this option was considered too complex for the current project scope.

Alternative 3: Layered Modular Architecture with External Service Integration— Chosen Approach

The selected approach is a layered modular architecture. The system is divided into clear layers and components:

Presentation Layer: React/Vite Web Interface

Application Layer: Symfony Backend API

Persistence Layer: PostgreSQL Database

Matching Layer: Python Deterministic Scoring Service

Support Layer: Private File Storage and Planned Cache

External Services: ASR, LLM API, Notification Service

The React frontend provides an accessible interface for PwD candidates, employers, administrators, and authorized verifiers. The Symfony backend API manages registration, authentication, role authorization, profiles, jobs, applications, verification, AI orchestration, and service integration. PostgreSQL stores confirmed persistent data, and the Python scoring service calculates deterministic compatibility results. Protected files use private storage. A cache will be added later for measured repeated non-sensitive reads, with expiry and invalidation rules.

External services are used only where they add clear value. ASR supports voice-to-text conversion, the LLM helps extract structured information from natural language, and the notification service sends alerts to users.

This architecture was selected because it is modular without being unnecessarily complex. It supports the Phase II requirements while remaining realistic for implementation.

Table 1: High-level architecture alternatives comparison

Criterion	Monolithic Application	Full Microservices	Chosen Layered Modular Architecture
Implementation complexity	Low	High	Medium
Maintainability	Medium	High	High
Scalability	Limited	Very high	Good
AI and scoring integration	Tightly coupled	Fully separated	Modular and manageable
Security management	Simple but centralized	Complex	Controlled and modular
Future extensibility	Limited	Very high	Good
Operational overhead	Low	Very high	Medium and proportionate to prototype scope
Component testability	Limited isolation	Strong isolation	Strong boundaries for scoring and AI/voice
Accessibility delivery fit	Shared interface possible	Shared interface may fragment across teams	One accessible React design system across all roles
Sensitive-data separation	Centralized in one application	Flexible but operationally complex	Explicit RBAC and private-storage boundary
FYP feasibility	Fast initially but difficult to extend safely	Low within the available time and infrastructure	High: implemented incrementally with clear future paths

The chosen architecture provides the best compromise between simplicity, modularity, accessibility, security, and future extensibility.

5.1.2 Applied architecture example

For example, replacing the speech-transcription provider affects the AI/voice adapter and configuration, while the Symfony authorization rules, PostgreSQL schema, and

deterministic scoring formula remain unchanged. If the AI/voice service is unavailable, the candidate can continue with manual profile entry. The evaluation in [Table 1](#) is illustrated by the independently running services in [Figure 6](#).

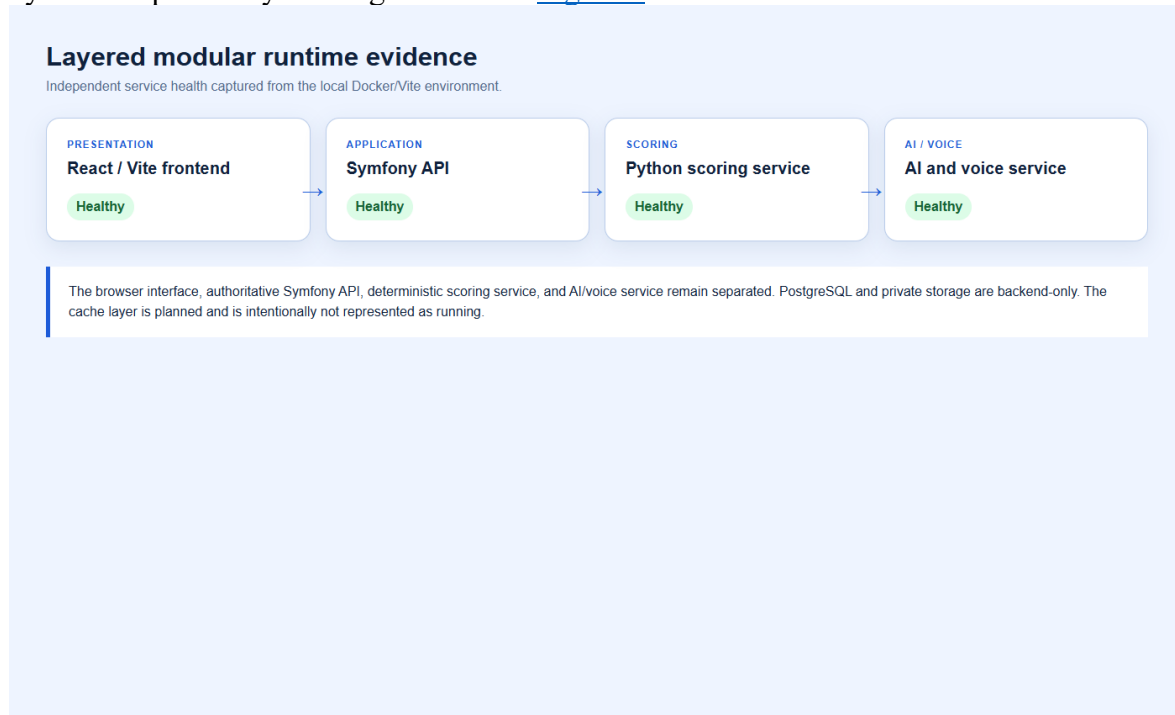


Figure 6: Layered modular runtime service boundaries and health evidence

5.1.3 Low level alternatives

At the component level, several design choices were also analyzed. These choices concern authentication, PwD candidate verification, multimodal input, AI-assisted NLP, matching logic, temporary storage, and caching.

A. Authentication and Role Management

The selected authentication approach is based on email and password authentication managed by the Symfony backend. Each user account is assigned a unique internal identifier, and the role field defines the type of account, such as PwD candidate, employer, administrator, or authorized verifier.

Public registration is limited to two roles: PwD candidates and employers. Administrator and authorized verifier accounts are not created through the public website; they are created and managed internally by the platform owner. This prevents unauthorized users from accessing sensitive administrative or verification functions.

Role-based access control is applied by the backend to protect platform features and data. For example, PwD candidates can complete their profile and apply to jobs, employers can manage job roles and applications, administrators can manage platform data, and authorized verifiers can update candidate verification status. This design keeps authentication simple, controlled, and aligned with the privacy and security requirements of the platform.

B. PwD Candidate Verification

Two alternatives were considered for verifying PwD candidates.

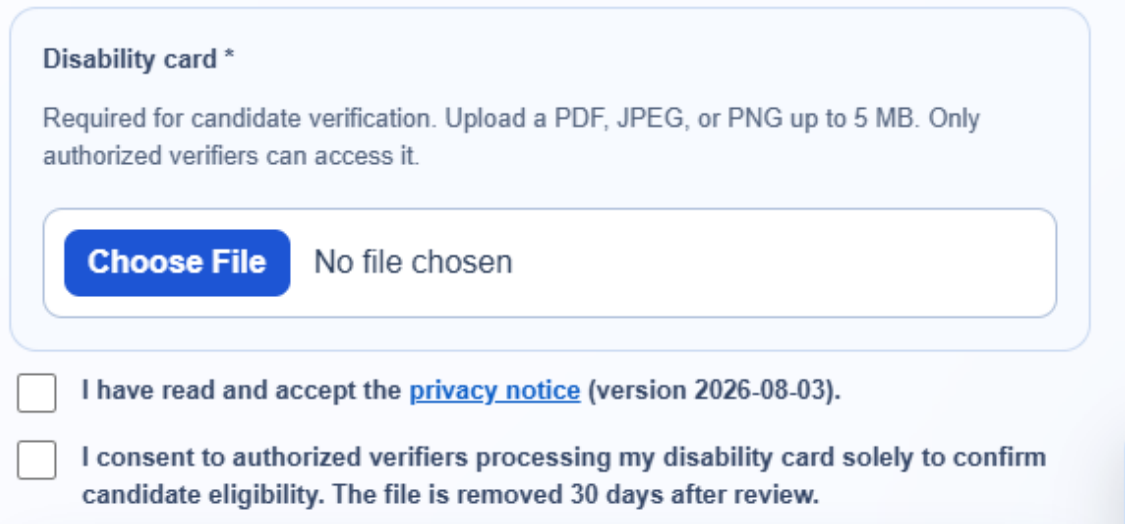
The first option was to store the uploaded disability card permanently in the system. This would allow the document to be reviewed later, but it would increase privacy and security risks because disability recognition documents are sensitive.

The chosen option is authorized human verification with private, time-limited document storage.

The candidate uploads a required disability card with explicit consent. The request remains pending until an authorized verifier approves or rejects it, and sign-in remains blocked until both email verification and document approval pass. The source file is scheduled for deletion 30 days after the decision; the verification status, reviewer, note, and audit events remain.

This choice reduces privacy risks and follows the principle of data minimization.

Applied verification example: a candidate submits a valid PDF and verifies the email address, but the account still cannot sign in while the review status is pending. Approval by an authorized verifier opens access; rejection keeps access blocked and requires a reason. The required upload and purpose-specific consent are shown in [Figure 7](#).



The screenshot shows a web form titled "Disability card *". Below the title, it states: "Required for candidate verification. Upload a PDF, JPEG, or PNG up to 5 MB. Only authorized verifiers can access it." There is a file upload area with a blue button labeled "Choose File" and the text "No file chosen". Below the upload area, there are two checkboxes with associated text:

- ☐ I have read and accept the [privacy notice](#) (version 2026-08-03).
- ☐ I consent to authorized verifiers processing my disability card solely to confirm candidate eligibility. The file is removed 30 days after review.

Figure 7: Candidate disability-card requirement and verification consent

C. Candidate Input Method

Three alternatives were considered for profile completion and job application forms.

The first option was manual form completion only. This is easy to implement but may create barriers for PwD candidates who have difficulty typing, navigating long forms, or using standard digital interfaces.

The second option was voice input only. This improves accessibility for some users, but it excludes users who prefer text or cannot use voice comfortably.

The chosen option is multimodal profile input with a complete manual fallback. The candidate can type directly or explicitly activate the microphone. The browser sends bounded audio to the AI/voice service, which returns an English, French, or Arabic transcript. The transcript remains editable and is not analysed until the candidate grants the required AI consent.

This approach supports accessibility while keeping the user in control.

D. AI-Assisted NLP Workflow

Two alternatives were considered for extracting useful candidate information from text.

The first option was a rule-based NLP system. It would detect information using predefined keywords and patterns. This approach is easier to control, but it is limited when users speak naturally, use mixed languages, or describe their abilities in different ways.

The chosen option is LLM-assisted NLP within the candidate profile workflow. After voice input is transcribed—or after the candidate types directly—the editable English, Arabic, or French text is analysed only after explicit consent. The protected service returns suggestions mapped to the current profile schema and controlled catalogues; it does not populate registration or application fields automatically.

The LLM returns a structured JSON output, for example:

```
{ "personalInformation": { "location": "Beirut" }, "educationLevel": "...", "disabilities": [], "skills": [], "preferences": [], "assistanceNeeds": [], "taskSkills": [], "unmatchedStatements": [] }
```

The structured JSON is displayed as editable suggestions rather than being applied automatically. Every suggestion starts unselected. The candidate chooses which items to confirm, and Symphony validates selected values against field limits and active catalogues before persistence. This keeps AI advisory and prevents unconfirmed sensitive information from becoming profile fact.

E. Matching Approach

Three matching alternatives were considered.

The first option was keyword-based matching between candidate profiles and job descriptions. This would be simple but not accurate enough because it does not evaluate the real tasks required by each job.

The second option was a fully automated ML decision system that directly decides if a candidate is suitable for a job. This could be powerful, but it would be less transparent and harder to justify ethically.

The chosen option is deterministic task-level ability-based matching. Each job is decomposed into weighted tasks with explicit feasibility assessments. The service compares confirmed candidate inputs with these task requirements, applies the approved mathematical policy, and returns the overall score and explainable task-level breakdown. It does not make the final recruitment decision.

This approach is transparent because the candidate receives the eligibility result, compatibility percentage, absolute compatibility band, and task-level explanations. The score represents compatibility with the configured vacancy under the documented policy; it is not an official disability percentage, employment decision, or guarantee of suitability.

Applied matching example: an employer selects Write, Read, and Count as important vacancy tasks. Their adjusted weights become 1.5 instead of the default 1.0. If assistance is available, a needs-assistance assessment earns the documented 0.75 feasibility factor; without assistance it becomes avoid. Explicit avoid remains zero and a mandatory avoid excludes the offer. [Figure 8](#) shows the controlled task selection and assistance setting.

JOB POSITION *
Positions are controlled by the platform administrator.

Bakery / Pastry Worker

LOCATION *

Beirut, Lebanon

APPLICATION DEADLINE *

mm/dd/yyyy

JOB TYPE *

Full-time

WORK MODE *

On-site

JOB DESCRIPTION *

IMPORTANT TASKS *
Select every catalogue task that is especially important for this vacancy. Important tasks receive greater scoring weight; at least one is required.

Search tasks...

Select all tasks Clear selection

Write x Read x Count x

✓ Write

✓ Read

✓ Count

○ Basic cooking knowledge to position

○ Keep minimum stock

3/189 catalogue tasks marked important

☒ Task assistance is available

Figure 8: Employer task-importance and assistance configuration

F. Storage of Voice, AI Suggestions, and Uploaded Documents

Two alternatives were considered for temporary data.

The first option was to store raw voice recordings, transcripts, AI suggestions, and uploaded documents in the database. This would make tracking easier, but it would store unconfirmed and sensitive information, which creates privacy risks.

The chosen storage boundary separates structured records from private files and transient AI data. PostgreSQL stores confirmed structured records and audit metadata. Profile audio remains in browser memory only long enough for transcription; unconfirmed suggestions remain transient; and disability cards or optional application documents use authenticated private storage with MIME/size validation, no-store responses, auditing, retention, and deletion controls.

This design supports privacy, security, and the human-in-the-loop principle.

Applied privacy example: candidate export returns the structured account data rather than raw disability-card bytes. Withdrawing AI consent prevents future AI analysis without deleting already confirmed profile facts. Password-confirmed account deletion removes the account and associated private application/verification documents. These user controls are shown in [Figure 9](#).

Privacy and your data

Review what the platform processes, download a copy of your data, or delete your account. Privacy notice version: loading.

Data we use

[Download my data \(JSON\)](#)

Disability-card retention

No verification request is linked to this account.

AI profile assistant

The platform records consent and operational events, but not the transcript text. You approve suggestions before profile changes are saved.

[Withdraw active AI consent](#)

Delete account

This permanently deletes your account and privately stored application and verification documents. This cannot be undone.

Password

Type DELETE

[Permanently delete my account](#)

[Voice navigation](#)

Figure 9: Candidate data-access, consent, retention, and erasure controls

G. Cache Layer

Two alternatives were considered for performance.

The first option was to avoid caching and always query the database or external services. This is simpler but less efficient when the same non-sensitive data is requested many times.

The selected future optimization is a cache layer for repeated non-sensitive data such as published jobs, tasks, and controlled vocabularies. It has not yet been implemented because the local performance baseline is within budget. The later implementation will be justified with deployed measurements and will define keys, expiry, invalidation, authorization boundaries, and failure fallback before caching is enabled.

This staged choice avoids premature complexity while retaining a clear path to improve deployed performance without caching disability documents, transcripts, unconfirmed AI output, tokens, or private candidate records.

Planned cache example: a key such as `catalogue:job-definition:{id}:v{version}` may store only published tasks and controlled vocabulary. An administrator catalogue import or job-definition update must invalidate the related version. Short TTL, cache-miss fallback to

PostgreSQL, metrics, and failure isolation are required. Tokens, disability cards, transcripts, unconfirmed AI output, private candidate profiles, and authorization decisions must never use this cache.

5.1.4 Low-level decision summary

The component alternatives, selected boundaries, and remaining trade-offs are summarized in [Table 2](#).

Table 2: Low-level component alternatives and selected boundaries

5.2	Decision	5.3	Alternatives evaluated	5.4	Selected boundary	5.5	Reason and remaining trade-off
5.6	Authentication	5.7	Server session; JWT API token; external identity provider	5.8	Symfony JWT plus server-confirmed role session	5.9	Fits the SPA/API baseline and explicit RBAC; staging now uses HTTPS and secret-managed JWT keys, while expiry/revocation governance and production monitoring remain required.
5.10	Candidate verification	5.11	Self-declaration; automated OCR/registry; authorized human review	5.12	Human review with private temporary evidence	5.13	Avoids unsupported automated eligibility decisions; creates reviewer workload and governance duties.
5.14	Candidate input	5.15	Manual voice only; multimodal	5.16	Manual form plus optional editable text/voice profile assistant	5.17	Maximizes access and graceful degradation; multilingual accuracy requires formal evaluation.
5.18	AI extraction	5.19	Rules; automatic persistence; reviewable suggestions LLM LLM	5.20	Consent-gated suggestions with human confirmation	5.21	Handles natural language while preventing unreviewed sensitive data from becoming fact.
5.22	Matching	5.23	Keywords; opaque learned decision; deterministic task scoring	5.24	Versioned deterministic weighted task policy	5.25	Explainable and testable; future calibration needs expert-reviewed examples rather than cosmetic score changes.

5.2 Decision	5.3 Alternatives evaluated	5.4 Selected boundary	5.5 Reason and remaining trade-off
5.26 Private files	5.27 Public upload; database blob; private filesystem/object store	5.28 Private storage locally and Supabase object storage in staging	5.29 Enforces authenticated access and deletion; staging uses purpose-specific Supabase buckets, while production encryption, backup, and erasure governance remain required.
5.30 Notifications	5.31 Email only; in-app only; both plus outcome log	5.32 Plain-text email and accessible in-app notice with minimized event log	5.33 Business decisions survive mail failure; production deliverability monitoring remains required.
5.34 Caching	5.35 No cache; process-local memory; shared Redis/equivalent	5.36 Measurement-driven shared cache planned	5.37 Add only with TTL, invalidation, privacy exclusions, observability, and database fallback.
5.38 Accessibility verification	5.39 Automated only; manual only; combined evidence	5.40 Automated scans/tests plus keyboard, zoom, reflow, and screen-reader plan	5.41 No AA claim until the remaining rendered/manual evidence is completed.

5.42

5.43 Solution modeling

In this section, the modeling of the selected solution is presented. After comparing the possible design alternatives, the chosen solution is described in more detail in order to prepare for the implementation phase. The following diagrams and flowcharts explain how the main components of the Phase II Inclusive Employment Platform interact internally and how they support the functional and non-functional requirements defined previously.

The solution model reflects the implemented modular structure: accessible React interface, Symfony API and data-access layer, PostgreSQL, Python scoring service, AI/voice service, email notifications, and private file storage. Caching remains a deferred optimization and is not shown as completed implementation evidence.

The accessible web interface supports keyboard navigation throughout public, candidate, employer, verifier, and administrator workflows. It includes a skip link, visible focus, logical focus movement, responsive controls, accessible form labels, dialog focus trapping and restoration, Left/Right arrow navigation where safe, and screen-reader-compatible outcome messages.

The implementation remains iterative: accessibility retests, representative-user feedback, deployment measurements, and new validated datasets may require controlled refinements. Current limitations and pending evidence are explicitly recorded instead of being presented as completed compliance.

5.43.1 Component Diagram

The component diagram shows the internal structure of the Phase II Inclusive Employment Platform. The accessible React web interface communicates with the backend API, which contains the main components for authentication, profile management, job and task management, ability catalog management, matching, applications, notifications, AI input processing, and data access.

The AI profile assistant manages an optional text/voice workflow. Voice is converted into an editable transcript, explicit consent is collected, and the internal extraction service returns structured profile suggestions. Suggestions remain unsaved until the candidate selects and confirms them. Page-aware action allowlists are used by the separate voice-navigation control.

The matching service compares confirmed candidate information with published job-task requirements and returns an explainable compatibility result. PostgreSQL stores confirmed platform data. Disability cards and application documents use authenticated private storage outside the public web root; audio is kept in browser memory for transcription and is not retained by the application. A cache layer remains part of the planned architecture and will be implemented later for measured, repeated non-sensitive reads with explicit expiry and invalidation rules.

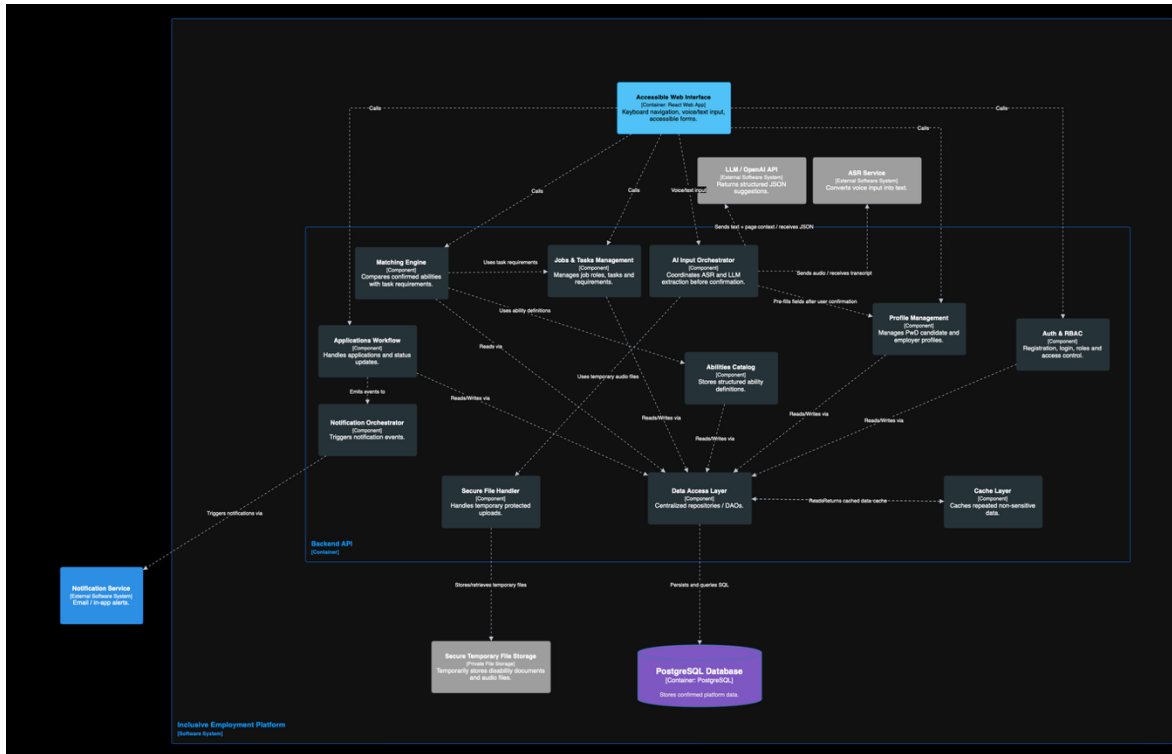


Figure 10: Component diagram

5.4.3.2 Required resources

Software Resources

The proposed solution is based on a web application stack composed of a React frontend, a Symfony/PHP backend API, a PostgreSQL relational database, and a Python/Flask matching microservice. The frontend communicates with the backend through HTTPS/JSON requests, while file uploads such as voice recordings or disability recognition documents are sent using secure multipart requests.

The backend exposes REST API endpoints and manages authentication, role-based access control, profile management, job and task management, applications, notifications, AI orchestration, and communication with the matching service. Stateless authentication can be implemented using JWT, while user roles are used to separate access between PwD candidates, employers, administrators, and authorized verifiers.

The frontend is responsible for the accessible user interface. It supports responsive pages, client-side validation, keyboard navigation, visible focus indicators, accessible forms, and role-based routing. It also supports multimodal input by allowing PwD candidates to complete certain forms using text or voice.

For AI-assisted input, an ASR service such as Whisper can be used to convert voice recordings into text. The transcribed text can then be sent to an LLM/OpenAI API to extract structured JSON suggestions. These suggestions are only saved after candidate confirmation.

For notifications, the platform can use in-app alerts and email notifications through Symfony Mailer, Mailtrap for testing, or SendGrid/SMTP for real-world delivery. A cache layer such as Symfony Cache or Redis can be used to store repeated non-sensitive data, such as job roles, tasks, abilities, and selected matching results.

Secure temporary file storage is required for protected uploads such as disability recognition documents and voice recordings during processing. These files should be stored outside public folders and deleted after verification or transcription when possible.

Hardware Resources

The core development can be carried out on standard laptops or PCs with at least 8 GB RAM, preferably 16 GB RAM when running Docker containers, the backend, frontend, PostgreSQL database, and matching service simultaneously.

Docker and Docker Compose provide the local development environment. Deployment and testing require a local or cloud host for the React production build, Symfony API, PostgreSQL database, Python scoring service, AI/voice service, notifications, and private file storage. The planned cache service can be added after deployed measurements establish its sizing, invalidation, and security requirements.

6 Implementation and Evaluation

This final-report draft records the implemented Phase II platform and verified evidence as of 8 September 2026. The work has progressed from a local prototype to an integrated and publicly reachable staging system. Each quantitative claim is linked to executable tests, machine-readable results, a deployed endpoint, or a clearly identified human-evaluation record; results that have not yet been supplied are marked as pending rather than inferred.

6.1 Implemented System

The platform uses a React/Vite frontend, a Symfony API, PostgreSQL persistence, a deterministic Python scoring service, and a separate AI/voice service. Docker Compose provides the reproducible local environment, while the staging deployment separates the static frontend, API, voice service, scoring service, managed database, and object storage. Dedicated candidate, employer, verifier, and administrator experiences are protected by server-validated role authorization.

Candidate registration has two independent eligibility gates: email ownership verification and authorized review of a mandatory disability card. Candidates can build and edit their profile, choose work, hospitality training, or both, select translated position interests, calculate explainable matches, and submit applications. Employers can publish work or training opportunities, load controlled catalogue tasks, configure education and practical abilities, highlight important tasks, and record assistance. Verifiers and administrators retain separate review and management boundaries.

6.2 Engineering Alternatives and Decision Rationale

The project decisions were evaluated against accessibility, explainability, privacy, security, implementation effort, maintainability, testability, expected prototype load, and future extensibility.

The selected design is not presented as universally best; it is the best fit for the current FYP scope and records the conditions under which it should evolve.

Table 3: Engineering design-decision matrix

Decision	Alternatives compared	Selected solution and rationale	Trade-off / follow-up
System architecture	Single monolith; full microservices; layered modular system	Layered modular system: simpler to deliver than full microservices while keeping scoring and AI/voice replaceable and independently testable.	Some services still share deployment; add operational separation only when scale justifies it.
Matching method	Keyword matching; opaque ML decision; deterministic task-level scoring	Deterministic task-level scoring: preserves the approved mathematics, exposes feasibility and weights, and is easier to audit ethically.	Future learned models require consented outcome data, versioning, evaluation, and stakeholder approval.
Candidate input	Manual only; voice only; multimodal	Manual form plus editable English, Arabic, or French typed/voice transcript: expands access without excluding users or making AI mandatory.	Formally evaluate Lebanese Arabic, code-switching, and assistive-technology interaction.
AI control	Automatic persistence; rule-based extraction; LLM suggestions with review	Consent-gated LLM suggestions with human confirmation: handles natural language while preventing unreviewed sensitive data from becoming fact.	External-service cost, latency, and extraction accuracy remain measurable risks; manual entry is the fallback.

Sensitive documents	Public uploads; database blobs; authenticated private storage	Private storage outside the web root with MIME/size checks, random names, auditing, no-store responses, and retention deletion.	Production backups, encryption/key management, legal retention, and incident response require operational policy.
Caching	No cache; immediate cache; measurement-driven staged cache	Planned measurement-driven cache for repeated non-sensitive catalogue reads, preserving a simple and correct baseline first.	Implement later with Redis or an equivalent only after deployed profiling; define TTL, invalidation, authorization, privacy exclusions, and fallback.
Accessibility verification	Informal UI review; automated-only scan; automated plus manual evidence	WCAG 2.1 AA-targeted engineering with automated scans, keyboard/reflow tests, and a required manual screen-reader/zoom retest.	No conformance claim until all pending rendered and assistive-technology evidence passes.

6.3 AI-Assisted Profile-Building Workflow

Candidates may keep using the complete manual profile form or provide an editable typed or spoken description. Speech transcription supports English, French, and Arabic selection. Audio is transcribed and discarded by the application. The transcript can be corrected before the candidate gives explicit consent for AI analysis.

The assistant returns structured suggestions with evidence and confidence. Every suggestion starts unselected and remains editable. Only explicitly selected values are sent to a confirmation endpoint, revalidated against active database catalogues, and persisted. This human-in-the-loop boundary prevents automatic diagnosis and preserves the approved scoring input contract.

OPTIONAL AI ASSISTANCE Human reviewed

Describe yourself in your own words

Speak or type in English, French, or Arabic. You edit the transcript and approve every suggestion before it is saved.

Spoken and written language

English Use microphone Audio is transcribed, then discarded by this application.

Your editable description or transcript

For example: where you live, your education, the tasks you can do, your experience, and your work goals.

☐ I agree to send this text to the AI assistant to create reviewable profile suggestions. It will not change my profile automatically. See the [privacy notice](#).

Create profile suggestions

0 / 4000 characters

Figure 11: Candidate AI-assisted profile-building workflow

Evidence: [AI profile workflow documentation](#)

6.4 Accessibility and Responsive Engineering

The public pages and all four role experiences were reviewed for responsive reflow, keyboard operation, focus order, semantic structure, accessible names, validation, contrast, reduced motion, dialogs, tables, filters, tabs, file controls, and live status messages. Left and Right Arrow provide requested focus navigation where this does not interfere with text fields or native widgets; Tab and Shift+Tab remain standard controls.

The remediation is substantial, but it is not represented as proof of complete WCAG 2.1 Level AA conformance. The final audit still requires a live post-remediation axe run, native 200% zoom, representative mobile and landscape tests, and manual NVDA/browser checks across authenticated workflows.

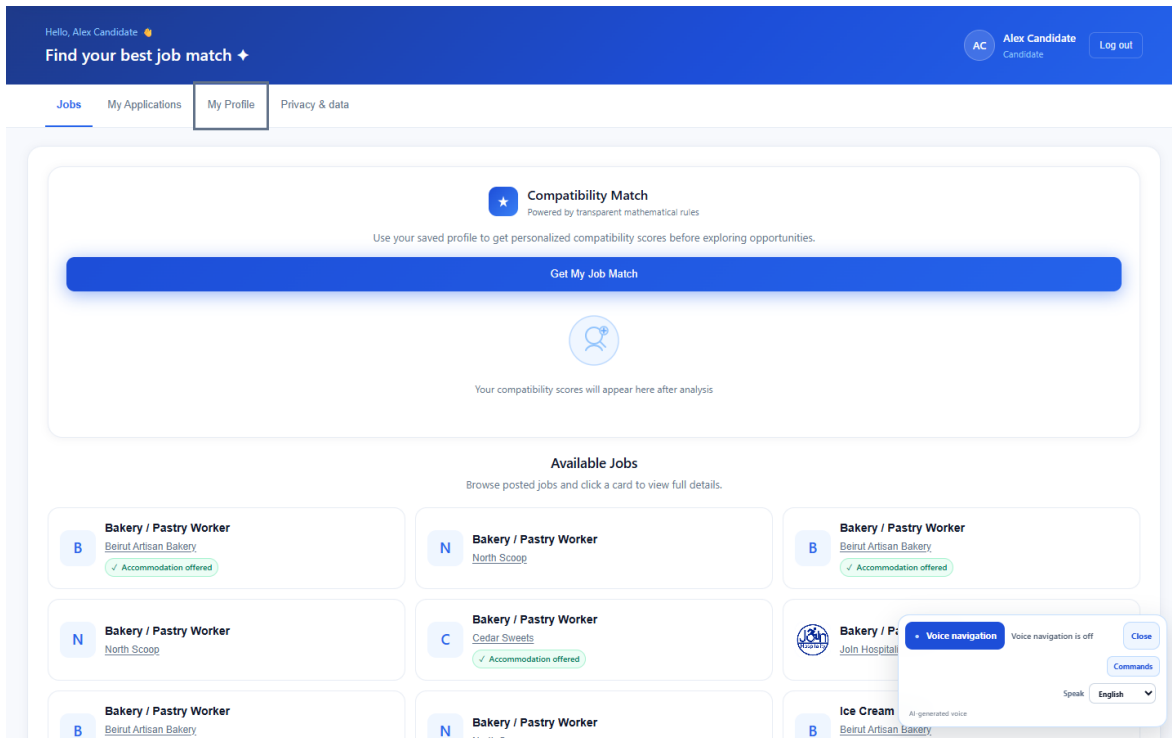


Figure 12: Keyboard/voice navigation and accessible notifications

Evidence: [WCAG 2.1 AA audit report](#)

6.5 Security, Verification, and Privacy

Backend firewalls protect role-prefixed APIs before controller execution, and the frontend confirms the active role through the protected session endpoint rather than trusting local token claims. Anonymous, forged-token, and cross-role access are rejected. Candidate verification decisions, document access, consent events, and application outcomes are auditable.

The candidate privacy dashboard explains stored data categories and consent history and supports structured export, AI-consent withdrawal, and password-confirmed account deletion. Technical safeguards support privacy principles but do not replace organizational legal review, incident response, backup-erasure rules, or production secret management.

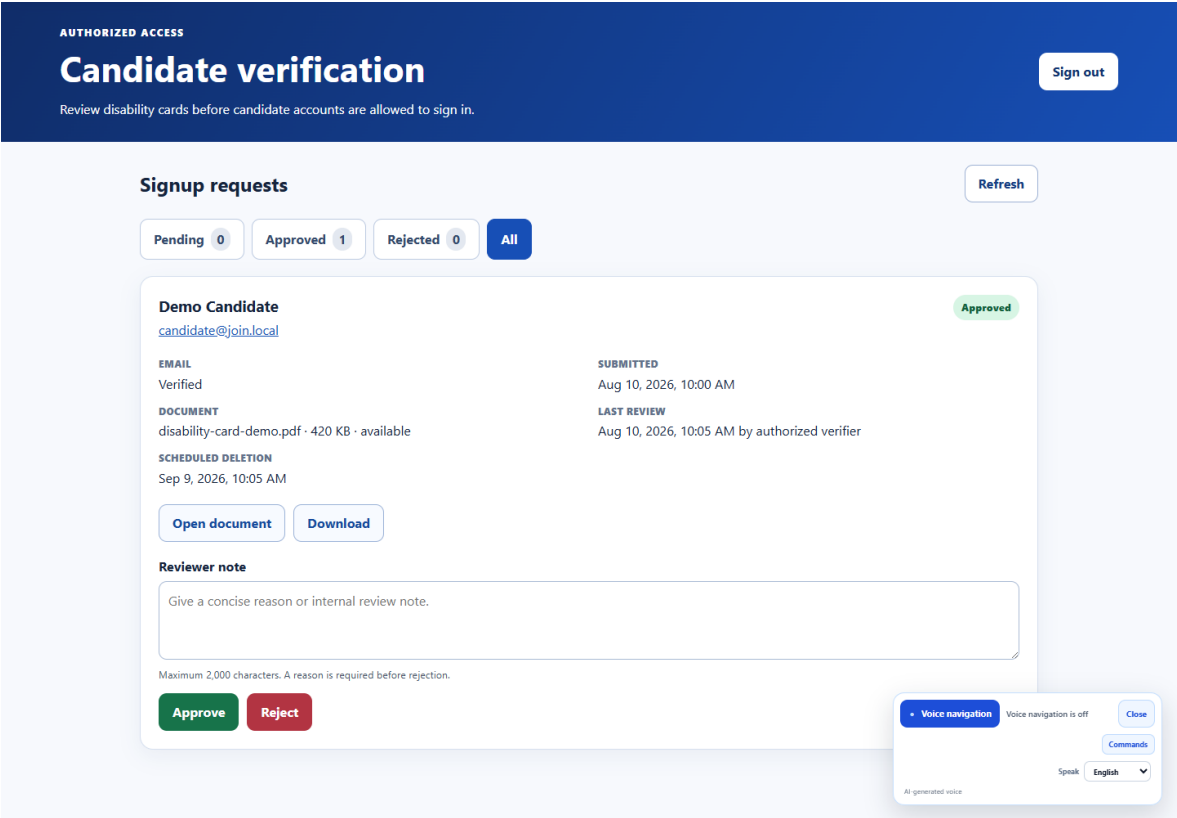


Figure 13: Authorized candidate-verification dashboard

Evidence: [candidate verification documentation](#)

Related evidence: [authorization security documentation](#)

Related evidence: [privacy controls documentation](#)

6.6 Catalogue and Scoring Policy

The controlled catalogue now contains 15 active hospitality job descriptions: the three original definitions plus twelve imported additions. The staging snapshot contains 923 ordered catalogue tasks, 16 disability categories, and 14,453 recorded disability-task assessments. Import validation preserves task order and source traceability, rejects unsupported feasibility values, and prevents duplicate or malformed definitions from silently changing the active catalogue.

The deterministic mathematical scoring policy was intentionally preserved and extended with explicit practical-ability and education rules. Matching uses confirmed candidate data, the selected work/training and position preferences, published opportunity requirements, task weights, mandatory/highlighted status, feasibility assessments, and offered assistance. AI suggestions remain advisory and cannot silently modify the official scoring contract.

6.7 Integration, E2E, and Performance Evidence

The repeatable verification suite passed 25 of 25 API integration checks and 26 of 26 browser end-to-end checks. The current deterministic scoring suite passed 17 of 17 tests and the voice-navigation/action-safety suite passed 52 of 52 tests. The production frontend build completed successfully, and the internationalization contract passed for three locales across seven namespaces.

The 4 September 2026 local production-build performance run passed every configured budget with zero violations. Recorded p95 API latency was 358.5 ms for public jobs, 144.0 ms for job definitions, 217.8 ms for the candidate session, and 132.3 ms for the candidate profile. Recorded page load/FCP p95 values were 56.5/168 ms for Welcome, 67.9/104 ms for Sign In, and 48.3/84 ms for Candidate. The largest recorded JavaScript and CSS assets were 150,516 and 6,050 gzip bytes. These values are regression evidence from the development computer, not a production-capacity claim.

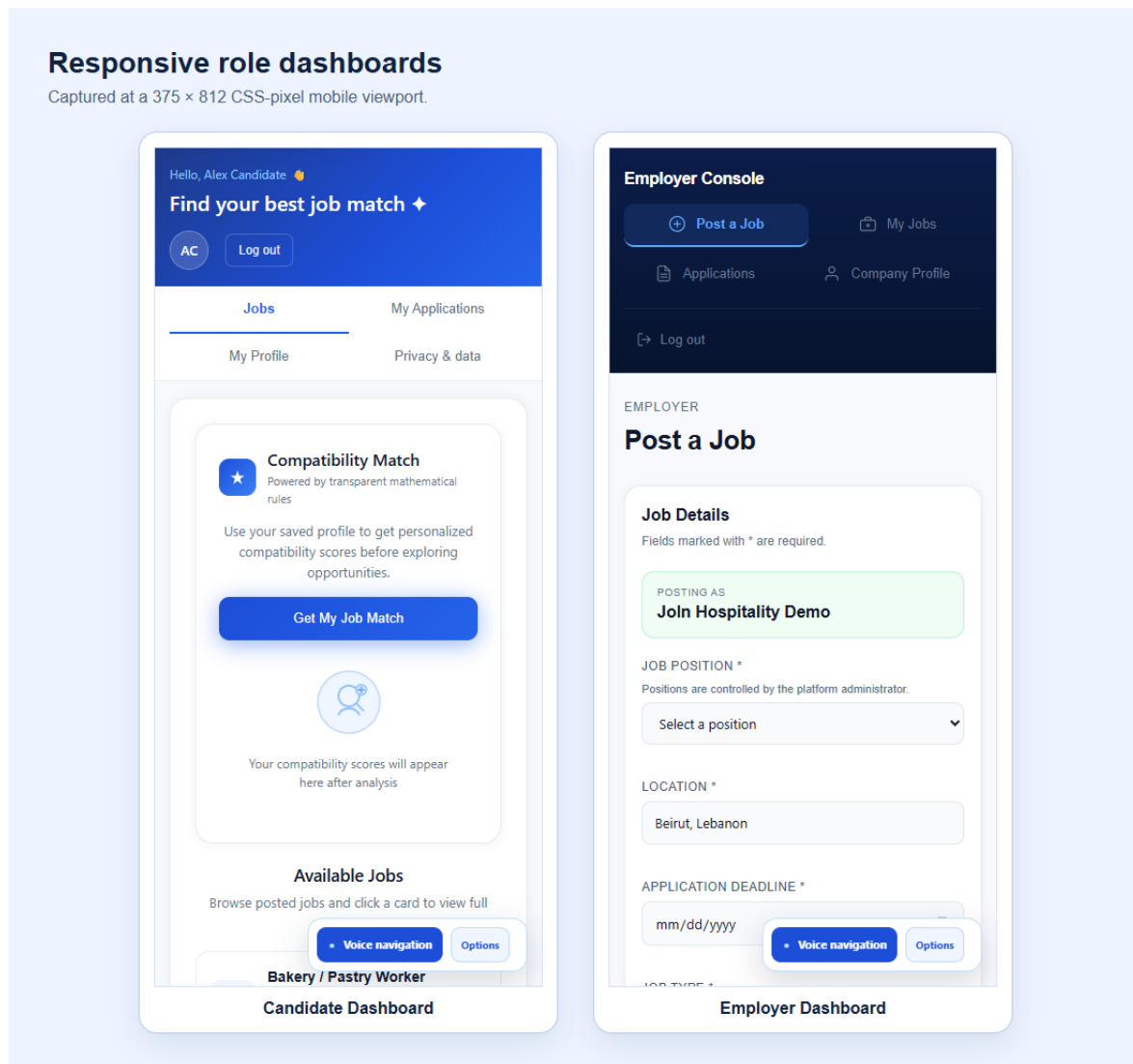


Figure 14: Responsive role dashboards

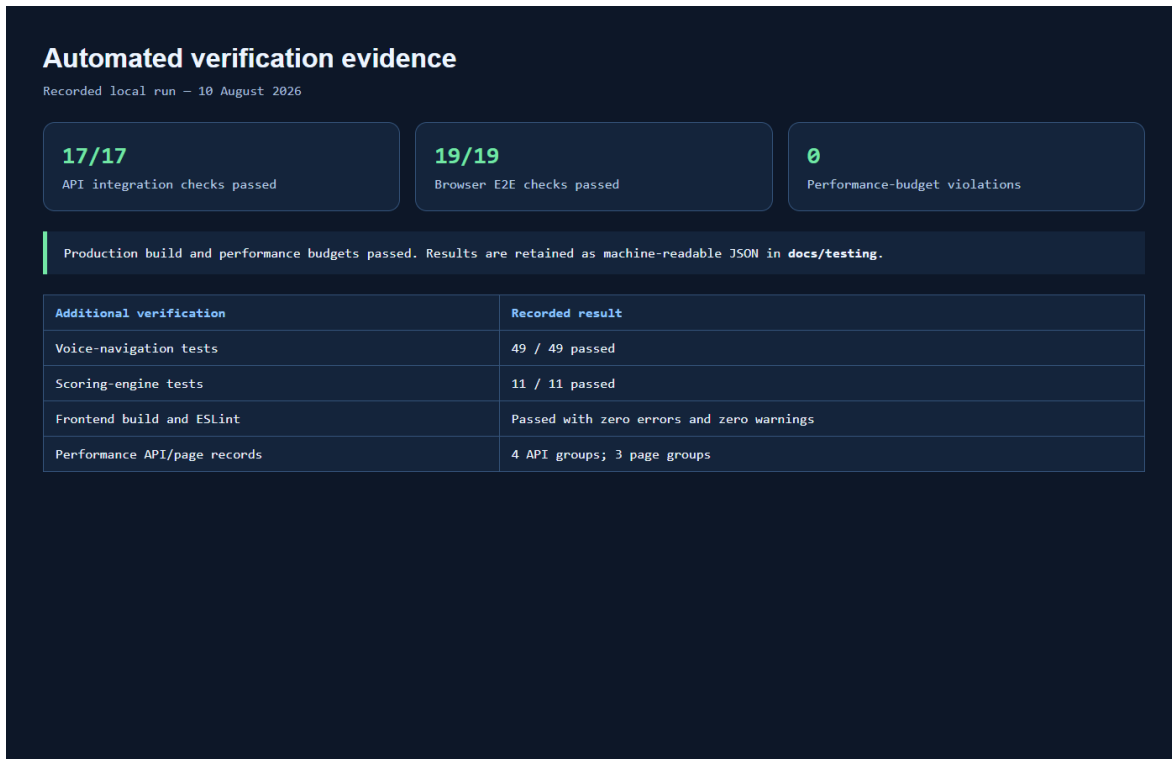


Figure 15: Integration, E2E, and performance evidence

Evidence: [integration, E2E, and performance report](#)

Machine-readable evidence: [performance results](#)

6.8 Evaluation Strategy

Evaluation uses triangulation rather than a single pass/fail result. Unit tests verify mathematical and voice-authorization rules; integration tests verify REST contracts and role isolation; browser E2E checks exercise public and authenticated journeys; the internationalization contract detects missing or inconsistent translation keys; the WCAG matrix combines axe, reflow, console, request, and keyboard evidence; performance measurements enforce explicit budgets; staging checks verify configuration and service boundaries; and representative-user sessions reveal usability barriers that automation cannot detect.

6.8.1 Automated Accessibility Result

The 4 September WCAG evidence matrix executed 104 route-scenario combinations across 13 public and authenticated routes and eight viewport or text scenarios. It recorded zero scan errors, axe-rule findings, horizontal-overflow scenarios, console-error entries, and failed-request entries. This is strong regression evidence, but it does not establish complete WCAG 2.1 Level AA conformance because automation cannot judge reading quality, announcement clarity, meaningful labels, cognitive effort, or successful use with assistive technology.

6.9 Cloud Staging Deployment and Operational Validation

The deployment objective was to prove that the complete platform could operate outside the development computer through a public HTTPS address and support evaluation from different devices and networks. A dedicated staging-deployment branch isolates cloud configuration from active feature development. The root `render.yaml` Blueprint defines one static frontend and three Docker web services: Symfony API, deterministic scoring, and voice/profile assistance. Dynamic services run in Frankfurt, the frontend is distributed through Render global static hosting, and Supabase in Frankfurt provides PostgreSQL persistence and object storage.

6.9.1 Deployment Phases

1. Environment isolation and infrastructure definition. A staging-specific Git branch and declarative Blueprint were created. Service paths, Dockerfiles, health endpoints, regions, free plans, frontend rewrites, CORS rules, and security headers are version controlled.
2. Managed data services. A Supabase project was created in Central EU (Frankfurt). PostgreSQL is accessed only by the backend through an SSL-enabled pooler connection. Storage uses private candidate-verification and application-document buckets plus a public employer-logo bucket; server-side MIME and size validation remains authoritative.
3. Secret and identity configuration. Database credentials, OpenAI access, Supabase server access, JWT keys and passphrase, the demonstration-account password, and independently generated scoring/profile service tokens were supplied through Render environment secrets. No secret value is stored in the Blueprint, frontend bundle, screenshots, or report.
4. Container hardening and service wiring. The backend image installs the catalogue-import runtime, generates JWT files from Base64 secrets, assigns the private key to the web-server group with restricted permissions, runs migrations, and starts the production server. Explicit service URLs and shared tokens constrain Symfony-to-Python calls, while the browser receives only public base URLs.
5. Data migration. The local PostgreSQL schema and demonstration records were exported, restored into Supabase, and checked through aggregate SQL and public/application endpoints. The verified staging catalogue contains 15 active datasets, 923 tasks, and 14,453 assessments. The public snapshot on 8 September contained three registered candidate accounts and 74 published work or training opportunities.
6. Functional validation. Health endpoints for the API, scoring, and voice services were checked; candidate, employer, verifier, and administrator demonstration accounts authenticated successfully; public catalogues loaded; candidate matches returned successfully after service warm-up; and the frontend communicated with all backend services through HTTPS.

6.9.2 Observed Deployment Failures and Corrections

The initial API deployment failed because the Supabase database password saved in Render did not match the newly generated project password. Replacing the `DATABASE_URL` with the confirmed pooler URI resolved PostgreSQL authentication. A second backend issue concerned generated JWT-key ownership: the runtime user could not read the private key. The entrypoint was corrected to assign the key to the web-server group and apply mode 640 to the private key. These failures and corrections are retained as

deployment evidence because they demonstrate diagnosis at the database, container, and operating-system permission boundaries.

6.9.3 Operational Result and Limitation

All four Render resources were deployed successfully and the Supabase-backed application became publicly reachable over HTTPS. The environment is suitable for demonstration and cross-device evaluation, but it is not production infrastructure. Free Render web services sleep after inactivity; an observed first matching request returned HTTP 503 while the scoring service was waking, whereas the same authenticated request returned HTTP 200 after warm-up. The demonstration procedure therefore warms health endpoints before use. Email transport is disabled, free-tier quotas apply, no availability or concurrency service level is claimed, and cache introduction remains conditional on measured need.

6.10 Human-Centred Accessibility Evaluation

An initial on-site evaluation was conducted at Arc en Ciel during the finalization phase. The reported session included interaction by a real candidate and an accessibility review with a blind participant who uses JAWS. This is important evidence because it tests the platform through lived interaction rather than only source inspection or browser automation. To avoid inventing results, the detailed participant profile, consent procedure, device and browser, JAWS version, assigned tasks, observed barriers, comments, completion outcomes, and resulting corrections remain marked for completion when the field notes are supplied.

[FIELD EVIDENCE TO COMPLETE] Record: session date and location; anonymized participant code and relevant access needs; informed-consent procedure; hardware, browser, JAWS version and settings; language; ordered task script; success/assistance/failure for each task; completion time; errors and recovery; participant comments; observer notes; severity and defect identifier; correction; and retest result.

6.10.1 Planned Quantitative Analysis

Each future session will use the same task definitions and evidence sheet. Primary measures are task-completion rate, independent-completion rate, assistance rate, error count, recovery count, completion time, and post-task ease rating. Secondary measures are issue count by severity, successful screen-reader announcements, navigation efficiency, transcription corrections, and participant satisfaction. Results will be reported by participant group and interaction mode so that a strong result from mouse users cannot hide a barrier for keyboard, voice, or screen-reader users.

For completion rates, the report will show the numerator and denominator together with the percentage. Time and error distributions will use median and interquartile range when the sample is small or skewed; mean and standard deviation may be added when appropriate. Before/after repeated measurements may use a paired non-parametric comparison such as Wilcoxon signed-rank only when the sample and protocol justify inference. Otherwise, the analysis will remain descriptive. Missing observations, assistance, abandoned tasks, and test-environment failures will be reported explicitly rather than removed.

[STATISTICAL RESULTS TO COMPLETE] Insert the final sample size, participant-group counts, task-level raw observations, formulas, descriptive tables, graphs, confidence intervals where defensible, identified defects, implemented corrections, and post-correction retest results.

6.10.2 Interpretation Boundary

A successful usability session shows that the tested participant completed the tested tasks under the recorded conditions; it does not prove accessibility for every disability, device, browser, language, or context. The final report will therefore separate automated conformance evidence, observed human usability, participant opinion, and unresolved limitations.

6.11 Requirements Traceability

Table 4: Requirements progress and verification matrix

Requirement area	Status	Implemented evidence	Remaining work
Authentication and RBAC	Implemented	Server-validated JWT sessions; explicit candidate, employer, verifier, and administrator route boundaries; anonymous, forged-token, and wrong-role checks.	Repeat security audit in the production deployment.
Candidate eligibility verification	Implemented	Mandatory disability-card upload and consent; email verification; authorized review; protected document access; approval/rejection audit.	Confirm institutional verifier policy and production retention schedule.
Candidate profile and AI assistance	Implemented	Manual form plus editable English/French/Arabic transcript; consent; reviewable unsaved suggestions; controlled-catalogue validation.	Evaluate Lebanese Arabic, mixed-language, and representative-user accuracy.
Task-level matching	Implemented	Approved deterministic scoring policy preserved; task weights, feasibility, mandatory status,	Validate future catalogue additions before import.

		assistance, and education remain explicit.	
Jobs, applications and outcomes	Implemented	Candidate, employer, and administrator workflows; optional application documents; accessible statuses; consent-aware outcome logging.	Collect sufficient real-world outcome data before any predictive-model study.
Accessibility and responsive design	Automated matrix passed; human evidence in progress	Keyboard workflows, focus management, semantics, contrast, reflow, reduced motion, errors, status announcements, and 104 automated route-scenario scans without recorded axe findings or overflow.	Complete the Arc en Ciel/JAWS evidence, native zoom, multi-device checks, corrections, and retests before a conformance statement.
Privacy and private storage	Implemented	Purpose-specific consent, MIME/size validation, private no-store downloads, audit events, export, consent withdrawal, deletion, and retention purge.	Complete jurisdiction-specific legal review, backup erasure, and incident-response procedures.
Integration and E2E testing	Implemented and passed locally	API integration: 25/25; browser E2E: 26/26; voice/action safety: 52/52; deterministic scoring: 17/17; production build and i18n contract passed.	Repeat the complete suites in CI and against staging; current staging health, role authentication, public data, and matching have been verified.
Performance baseline	Local budgets passed; staging behavior observed	Zero local budget violations. API and page p95 values are recorded; staging exposed a free-service cold-start delay rather than a scoring defect.	Measure deployed warm/cold response, multiple devices/networks, and bounded concurrency before making a capacity claim.

Dataset expansion to 15 roles	Implemented technically	15 active descriptions, 923 tasks, 16 disability categories, and 14,453 assessments are deployed in staging.	Complete domain-stakeholder validation, translation review, and regression retesting after approved dataset corrections.
Cache layer	Deferred by measurement	The non-cached local baseline passes its budgets; sensitive-data exclusions and the decision boundary are documented.	Introduce a shared cache only if deployed profiling identifies a justified repeated-read bottleneck.

6.12 Current Limitations

The following items remain open and are not represented as completed: detailed analysis and documentation of the Arc en Ciel/JAWS field session; a larger representative usability sample; native-device and cross-network retesting; formal multilingual transcription accuracy statistics; deployed-origin performance and concurrency measurements; production monitoring, backups, email delivery, retention automation, incident response, and privacy/legal governance. Redis or another cache remains deferred unless profiling demonstrates a justified need.

7 Conclusion, Limitations, and Future Work

Phase II delivered an integrated platform that advances the Inclusive Web Platform from a limited ability-matching proof of concept to a deployed, multilingual, accessible, and reviewable hospitality opportunity system. The principal contributions are candidate eligibility verification, private document handling, server-enforced role authorization, work/training preferences, translated position interests, controlled employer requirements, editable multilingual transcription, consent-gated AI suggestions, deterministic explainable matching, responsive keyboard and voice interaction, privacy controls, outcome logging, a fifteen-description catalogue, cloud staging, and repeatable engineering evidence.

The strongest project design decision is to keep people in control. AI output is advisory and editable, candidate confirmation is mandatory before persistence, disability documents are separated from employer views, and the matching score supports—not replaces—human recruitment decisions. The mathematical scoring contract has not been silently altered by the AI workflow.

The final prototype is not presented as proof of regulatory compliance, clinical suitability, production availability, or employment effectiveness. It is an engineering result supported by reproducible automated tests, a working cloud staging environment, and emerging representative-user evidence. The remaining evaluation tasks are explicitly separated so reviewers can reproduce the evidence and understand the limits of each claim.

7.1 Remaining Validation and Future Work

1. Complete the human accessibility evidence: transcribe the Arc en Ciel field notes, document the JAWS configuration and task outcomes, resolve any defects, and perform a recorded retest. Repeat native 200% zoom and keyboard-only checks on representative deployed workflows before making any conformance statement.

2. Obtain formal domain-stakeholder validation for the twelve additional hospitality job descriptions, their task decomposition, feasibility assessments, and translation quality. Preserve dataset versions and rerun catalogue/scoring regression tests after every approved change.
3. Extend representative usability testing across candidates with different access needs, employers, and authorized verifiers. Apply the common protocol and calculate task completion, independence, assistance, time, errors, recovery, and satisfaction statistics without hiding missing or unsuccessful observations.
4. Evaluate English, French, Arabic, Lebanese Arabic, accents, noise, and mixed-language transcription/extraction using consented samples. Report correction rate and failure cases, and retain manual entry as the operational fallback.
5. Repeat integration, E2E, security, accessibility, and performance suites against staging from multiple devices and networks. Add bounded concurrency measurements, uptime/error monitoring, email testing, retention scheduling, backup/erasure procedures, incident response, and formal privacy review before any production proposal.
6. Keep caching measurement-driven. Introduce a shared cache only if deployed profiling identifies repeated non-sensitive reads that materially affect response time; then define TTL, invalidation, authorization, privacy exclusions, observability, and database fallback before enabling it.
7. Replace the highlighted field/statistical placeholders with approved evidence, review screenshots for readability and privacy, update captions, cross-references, contents, and page numbers, and complete supervisor review before submission.

7.2 Risk Management

Data quality remains the main scoring risk. The project mitigates it through schema-validated imports, source references, versioned datasets, explicit missing-assessment behavior, and deterministic regression tests. The twelve newly imported descriptions expand coverage but still require formal domain-stakeholder validation. AI/ASR risk is controlled through editable transcripts, consent, human confirmation, server validation, and a complete manual fallback.

Privacy and authorization risks are reduced through role-prefixed API rules, server-confirmed sessions, private storage, random filenames, server-side MIME validation, no-store responses, document access auditing, retention scheduling, data export, consent withdrawal, and account deletion. Production governance, backups, secrets, incident response, and legal validation remain organizational responsibilities.

Performance and schedule risk are managed through versioned source control, small testable increments, explicit automated budgets, machine-readable results, and documented limitations. A cache should be introduced only after deployed evidence identifies a bottleneck, since premature caching would add invalidation and privacy risk.

7.3 Project Management

Development evidence is version controlled through focused commits and technical documents stored beside the source. Cloud deployment is isolated on the staging-deployment branch and defined by render.yaml. Requirements, security decisions, privacy controls, accessibility remediation, dataset imports, tests, machine-readable results, and operational limitations are retained for traceability.

Project repository and deployment branch: inclusive-web-platform — staging-deployment

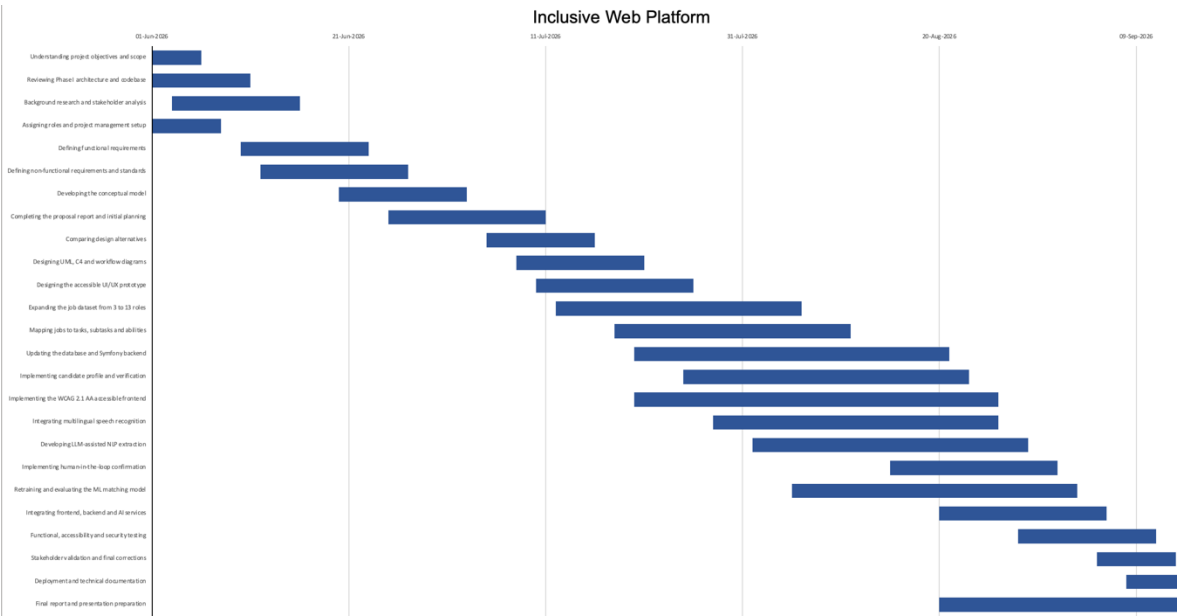


Figure 16: Gantt chart

8 References

[1] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2025.

[2] W3C Web Accessibility Initiative, “WCAG 2 Overview,” W3C.

[3] European Parliament and Council of the European Union, “Regulation (EU) 2016/679, General Data Protection Regulation,” EUR-Lex, 2016.

[4] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2011 — Systems and software engineering — Life cycle processes — Requirements engineering,” ISO, 2011.

[5] ISO/IEC, “ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models,” ISO, 2011.

[6] OWASP Foundation, “OWASP Web Security Testing Guide.”

[7] Symfony, “Security,” Symfony Documentation.

[8] Symfony, “Symfony Documentation.”

[9] React, “Accessibility,” React Documentation.

[10] OpenAI, “Structured model outputs,” OpenAI API Documentation.

- [11] OpenAI, "Introducing Structured Outputs in the API," OpenAI, 2024.
- [12] Université Saint-Joseph, Moodle platform.
- [13] International Organization for Standardization, ISO 9241-110:2020 — Ergonomics of human-system interaction — Part 110: Interaction principles, 2020.
- [14] Z. Mansour and C. Rizk, Inclusive Web Platform source code and technical documentation, GitHub, branch staging-deployment, 2026.
- [15] Render, "Blueprint Specification," Render Documentation, 2026, <https://render.com/docs/blueprint-spec>.
- [16] Render, "Deploy for Free," Render Documentation, 2026, <https://render.com/docs/free>.
- [17] Supabase, "Database Connections" and "Storage," Supabase Documentation, 2026, <https://supabase.com/docs>.
- [18] Freedom Scientific, "JAWS Screen Reading Software," 2026, <https://www.freedomscientific.com/products/software/jaws/>.

9 Appendix A — System Architecture, Setup, and Operations

9.1 Implemented Runtime Architecture

The final prototype follows the selected layered modular architecture. React renders user experiences; Symfony remains the authoritative API, authorization, and business-workflow boundary; PostgreSQL stores confirmed records; Python services isolate deterministic scoring and AI/voice functions; and object storage separates private evidence from public assets. Docker Compose reproduces the local stack, while Render and Supabase provide the staging topology. A cache is deliberately excluded because current evidence has not established a need.

Table 5: Runtime services and local endpoints

Component	Technology	Local endpoint	Responsibility
Frontend	React 19 and Vite	http://localhost:5173	Accessible public pages and candidate, employer, verifier, and administrator interfaces.
Backend API	Symfony and PHP	http://localhost:8081/api	Authentication, authorization, persistence, business workflows, file access, mail, and service orchestration.
Database	PostgreSQL 16	Docker internal service	Authoritative accounts, profiles, jobs, tasks, assessments, applications, outcomes, consents, and audits.

Scoring service	Python HTTP service	<code>http://localhost:5001</code>	Deterministic task-level eligibility, compatibility, explanation, and ranking. Speech transcription, bounded voice interpretation, speech feedback, and protected profile extraction. Disability cards and optional application files with authenticated access and retention controls. Repeated non-sensitive catalogue reads with TTL, invalidation, monitoring, and database fallback.
AI and voice service	Python/Flask and OpenAI adapters	<code>http://localhost:5002</code>	
Private storage	Symfony var volume	Not publicly routed	
Planned cache	Redis or equivalent after measurement	To be defined	

9.2 Prerequisites and First-Time Setup

Required development software is Git, Docker Desktop with Docker Compose, Node.js with npm, and a recent browser. The repository should be cloned, the backend environment files should be prepared from their examples, and strong local values should be supplied for JWT, database, profile-AI, mail, and OpenAI configuration. Real secrets must never be committed or exposed through Vite variables.

Backend startup command: `cd backend; docker compose up -d --build`. First initialization then runs composer install, JWT key generation, Doctrine migrations, and—only for controlled development—fixtures. Fixture loading purges and recreates demonstration data; it must never be used against valuable or production records.

Frontend startup command: `cd frontend; npm.cmd install` when dependencies are absent, followed by `npm.cmd run dev -- --host 0.0.0.0`. The host option allows testing from another device on the same trusted local network. Frontend API and voice base addresses must use the development computer LAN address rather than 127.0.0.1 when a phone is the client.

9.3 Daily Operation, Validation, and Troubleshooting

Daily startup uses `docker compose up -d` in backend and `npm.cmd run dev` in frontend. `docker compose ps` and service health endpoints confirm readiness. Logs are inspected with `docker compose logs --tail 100` or `docker compose logs -f` followed by php, database, scoring-engine, or voice-navigation.

Database changes require a versioned Doctrine migration followed by `docker compose exec php php bin/console doctrine:migrations:migrate --no-interaction` and `doctrine:schema:validate`. Backend source is bind-mounted, but container rebuild or recreation is required after Dockerfile, dependency, or environment changes. PostgreSQL data persists in a named volume unless `docker compose down -v` is deliberately executed.

The standard verification sequence includes Symfony configuration/schema checks, scoring unit tests, voice unit tests, frontend build and ESLint, API integration, browser E2E, performance

budgets, the security route audit, and the documented WCAG browser/manual matrix. Common failures are diagnosed by checking the current directory, container state, port mapping, CORS origin, migrations, dependencies, fixture credentials, service health, and environment configuration in that order.

10 Appendix B — Authentication, Candidate Verification, Security, and Privacy

10.1 Authentication and Authorization Model

Public registration creates candidate or employer accounts only. Administrator and verifier authority is assigned internally. Symfony authenticates protected API calls before controllers run. Route groups require `ROLE_CANDIDATE`, `ROLE_EMPLOYER`, `ROLE_VERIFIER`, or `ROLE_ADMIN` as appropriate; administrators inherit verifier API oversight but do not enter the verifier frontend route. Public exceptions are limited to registration, login, email verification, password recovery, published jobs, public job definitions, documentation, and CORS preflight.

Frontend route protection does not trust decoded browser claims. Before a dashboard renders, the application calls the protected session endpoint and compares the server-confirmed role with the requested route. Validation repeats periodically and when focus returns to the browser. Tokens use `sessionStorage` rather than persistent `localStorage` and are never placed in protected document URLs.

10.2 Candidate Eligibility Verification

Candidate access uses two independent gates: ownership of the registered email address and approval of an uploaded disability card by an authorized verifier. Candidate registration requires a PDF, JPEG, or PNG document up to 5 MB and explicit consent to verification processing. The two decisions may occur in either order, but a new candidate cannot sign in until both pass. Employer registration requires email verification but no disability document.

The verifier dashboard lists pending, approved, rejected, or all requests; presents email-verification status; opens or downloads the protected file through an authenticated request; records approval or rejection with reviewer identity, note, and timestamp; and requires a clear reason for rejection. Email notification failure does not roll back the saved decision and is reported separately.

10.3 Private Storage, Consent, and Candidate Rights

Disability cards and optional application documents use cryptographically random stored names below `backend/var/private`. Server-detected MIME type—not filename extension—controls acceptance. Empty or oversized files are rejected. Authorized responses use private, no-store caching and `X-Content-Type-Options: nosniff`. Every card view or download records actor, action, request, and timestamp.

A verification decision schedules the source card for deletion after 30 days while retaining the status and audit record. The purge command must run at least daily. Candidate account deletion removes associated private card and application files before database records are removed.

The candidate Privacy & data dashboard explains stored categories and consent history and provides JSON export, withdrawal of future AI analysis consent, and password-confirmed account deletion. These controls support data minimization, purpose limitation, transparency, retention,

access control, portability, consent withdrawal, and erasure; they do not independently prove compliance with every privacy law.

Table 6: Security and privacy control summary

Control area	Implemented control	Evidence or behavior	Remaining operational requirement
Authentication	Email/password with secure hashing and JWT	Protected APIs require X-Auth-Token; query-string token extraction is disabled.	Production secret rotation, monitoring, and secure transport.
Authorization	Explicit route-prefix RBAC	Candidate, employer, verifier, and administrator roles cannot use protected APIs assigned to other roles.	Repeat route audit after every new endpoint and in deployment.
Candidate eligibility	Email verification plus authorized disability-card review	Both independent gates must pass before a newly registered candidate can sign in.	Institutional verifier assignment and review policy.
Private uploads	Private path, random filename, MIME and size validation	Files are outside the public web root and returned only through authenticated no-store responses.	Backup encryption/erasure and production key management.
Retention	Card deletion scheduled 30 days after a decision	Status remains after the source file is purged; access and decisions are audited.	Run the purge command daily and approve jurisdiction-specific retention.
Candidate control	Consent history, export, withdrawal, deletion	Privacy dashboard supports JSON export, AI-consent withdrawal, and password-confirmed account deletion.	Formal legal review, controller contact, incident response, and backup erasure.
AI minimization	No automatic diagnosis or persistence	Audit metadata excludes transcript, narrative, raw audio,	Provider agreement and deployment privacy notice for external AI processing.

		passwords, and private documents.	
--	--	--------------------------------------	--

10.4 Security Verification and Deployment Duties

The route regression audit checks anonymous calls, forged JWTs, cross-role attempts, valid-role access, and intentionally public catalogue endpoints. Every new protected endpoint must be placed under the correct prefix or explicit access rule and added to this audit. Production responsibilities still include HTTPS, secret rotation, rate limiting where appropriate, backup protection and erasure, monitoring, incident response, verifier governance, and jurisdiction-specific legal approval.

11 Appendix C — AI-Assisted Profile Building and Voice Navigation

11.1 Candidate Profile Assistant

The AI profile assistant is optional. The manual form remains fully usable if AI or speech services are unavailable. A candidate chooses English, French, or Arabic, types a narrative or records speech, edits the resulting transcript, reads the privacy notice, and explicitly consents before requesting suggestions. Audio is held in browser memory for transcription and discarded by this application.

The extraction service proposes structured personal information, education, active-catalogue disabilities, skills, preferences, assistance needs, and active task skills. Each item includes evidence and confidence where available. Every suggestion starts unselected; the candidate can edit, select, ignore, or reject it. Only selected values reach the confirmation endpoint, where Symphony revalidates field lengths and controlled-catalogue membership before persistence.

The assistant may suggest a disability only when it was stated explicitly and exactly matches the active catalogue. It does not diagnose or infer medical facts. Unmatched useful text is displayed for manual review and is not automatically stored. Audit events retain language, consent/outcome type, character count, and item counts—not audio, transcript, or narrative content.

Confirmed task skills enrich the profile but are not sent to the scoring engine. This invariant protects the approved mathematical input contract. Introducing AI-derived fields into scoring would require a separately versioned, reviewed, migrated, and regression-tested policy.

11.2 Voice-Navigation Safety Architecture

Voice navigation follows the principle: flexible understanding, restricted choices, deterministic execution. Speech transcription and language interpretation may be probabilistic, but model output is schema constrained and must pass a trusted registry. The model cannot generate routes, selectors, JavaScript, arbitrary clicks, or unregistered operations. React executes only a fixed authorized action returned by the registry.

The voice lifecycle covers off, permission request, listening, processing, speaking, paused, and error states. A bounded utterance is recorded, silence ends the turn, the selected language and current page context are supplied, and no more than six structured history turns are retained in browser memory. The registry returns authorized, clarification, or rejected. Clarification choices are revalidated, bare unanchored confirmation is rejected, and disabling voice clears history and releases microphone resources.

Registered actions include trusted route navigation, browser back, bounded scrolling, page-section reading, internal dashboard views, authentication-form fields and confirmation-gated submission, candidate profile and matching actions, employer/administrator operations, dynamic records already loaded by the application, and grounded questions about bounded readable content on the current page. Voice is always optional and every function retains a keyboard/mouse equivalent.

11.3 Language, Privacy, Configuration, and Limitations

The profile assistant exposes English, French, and Arabic. The voice-navigation command subsystem currently provides explicit English and Arabic selection; French command coverage is a remaining requirement even though French profile transcription is available. Formal accuracy evaluation is still required for French, Arabic, Lebanese Arabic, accents, code-switching, noise, and representative speech patterns.

Audio and transcript content are sent to the configured OpenAI services when the user activates the relevant function. API keys remain server-side and are excluded from Git and Docker build context. CORS, file sizes, transcript lengths, feedback lengths, history size, and schema fields are bounded. Website-question support may send bounded readable page content but does not collect input values; authenticated rendered text may nevertheless contain personal data and therefore requires a clear production privacy notice.

The principal internal endpoints are POST /api/profile/transcribe, POST /api/profile/extract, POST /api/voice/process, POST /api/voice/interpret, and POST /api/voice/speech. Symphony exposes authenticated POST /api/candidate/profile/ai-suggestions and /api/candidate/profile/ai-confirm. The internal extraction endpoint requires the shared PROFILE_AI_TOKEN.

12 Appendix D — Deterministic Scoring Engine and Catalogue Policy

12.1 Authoritative Data Flow

React requests matches but never calculates the official score. Symphony authenticates the candidate, loads authoritative PostgreSQL profile and offer data, constructs the scoring payload, and calls the Python service. Python validates the domain objects, applies eligibility and mathematical rules, returns explanations and ranking, and Symphony adds presentation metadata before React renders the result.

12.2 Mathematical Model

For task i , let w_i be its positive base weight, h_i its highlighted multiplier, $a_i = w_i \times h_i$ its adjusted weight, f_i its effective feasibility factor, and $p_i = a_i \times f_i$ its earned points. For an eligible offer, $P_{\max} = \sum a_i$, $P_{\text{earned}} = \sum p_i$, and $\text{Score} = \text{round}(100 \times P_{\text{earned}} / P_{\max}, 2)$. Valid jobs contain at least one positive-weight task, so division by zero is excluded by domain validation.

Table 7: Deterministic scoring policy

Policy element	Current value	Effect	Engineering rationale
Base task weight	1.0 by default	Defines task importance before vacancy highlighting.	No importance is guessed from wording or spreadsheet order.
Highlighted multiplier	1.5	Employer-selected important tasks influence numerator and denominator.	Represents the real vacancy while keeping the numeric rule centralized.
Feasible factor	1.00	Earns the complete adjusted task weight.	Represents independent feasibility in the supplied assessment.
Needs-assistance factor	0.75 when assistance exists; otherwise avoid	Earns partial points only when the offer provides assistance.	Explicit proof-of-concept policy, not a medical probability.
Avoid factor	0.00	Earns no points; mandatory avoid excludes the offer.	An explicit avoid assessment cannot be rescued by assistance.
Missing assessment	Feasible fallback plus assumption flag	Avoids treating unchecked structural cells as negative evidence.	The returned explanation identifies that the value was assumed.
Multiple disabilities	Most restrictive assessment wins	Uses the minimum effective feasibility across confirmed disabilities.	Conservative, deterministic, and explainable.
Score	$100 \times \text{earned adjusted weight} / \text{maximum adjusted weight}$	Rounded to two decimals for eligible offers.	A null score means a hard eligibility gate failed; zero has a different meaning.

12.3 Eligibility, Ranking, and Interpretation

Education and mandatory-task rules are hard gates. If a gate fails, eligible is false and score is null; null must not be confused with an eligible score of zero. For multiple confirmed disabilities, the most restrictive assessment for each task wins. Assistance can transform needs-assistance into the 0.75 factor but never overrides explicit avoid.

Eligible offers are sorted by descending score, followed by title for ties; excluded offers follow eligible ones. Highest-ranked is relative to the returned pool and does not mean strongly

compatible. Presentation bands are strong 80–100, moderate 60–below 80, limited 40–below 60, and low below 40. These labels do not alter the formula or authorization to apply.

12.4 Catalogue Baseline and Missing Assessments

The active catalogue contains 15 hospitality datasets. The staging snapshot includes 923 ordered tasks, 16 disability categories, and 14,453 recorded assessments. The three original workbooks and twelve imported descriptions are normalized through the same schema, preserving source references and explicit feasibility values. The added datasets improve coverage but remain subject to final domain-stakeholder validation.

At runtime, absence of an assessment is treated as no recorded restriction, using the documented feasible fallback and returning an assumed-from-missing-data flag in the explanation. This policy follows the supplied source interpretation and prevents unchecked structural cells from being converted into invented negative evidence. Future stakeholder review may revise the policy through a versioned change, but the current mathematical approach is intentionally preserved.

12.5 Determinism, Testing, and Future Calibration

The score depends only on candidate education and confirmed disabilities; job education, assistance, tasks, weights, mandatory/highlighted status; assessment data; and central policy constants. There is no randomness, training, timestamp input, external web data, or browser-side official calculation. Company description, location, deadline, and prose cannot alter the score.

The current seventeen deterministic unit tests cover all-feasible scoring, assistance on/off behavior, explicit and mandatory avoid, optional avoid, missing-assessment fallback, multiple disabilities, education exclusion, highlighted weighting, practical-ability rules, application-only position knowledge, serialization, and invalid weights. Future calibration must use expert-reviewed candidate/opportunity examples and expected outcomes rather than cosmetically changing percentages.

13 Appendix E — Responsive Design, Keyboard Access, and WCAG Evidence

13.1 Audit Scope and Implemented Corrections

The accessibility scope includes Welcome, Employers, Privacy, Voice Help, Sign Up, Sign In, password recovery/reset, candidate setup and dashboard states, AI profile review, jobs and applications, privacy controls, employer workflows, verifier review, administrator management, file controls, dialogs, filters, tabs, notifications, and shared navigation. Responsive checks target 320, 375, 768, 1024, and 1440 CSS pixels, landscape orientation, text spacing, and 200% zoom.

Implemented corrections standardize containers, spacing, typography, controls, breakpoints, wrapping, table/card behavior, and dialog sizing. Shared accessibility work covers alternative text, headings and landmarks, accessible names/descriptions, focus visibility, focus management, error identification, live status announcements, contrast, reduced motion, semantic HTML, and valid ARIA. Candidate, employer, verifier, and administrator management operations remain keyboard reachable.

Table 8: Accessibility verification coverage

Area	Implemented evidence	Relevant criteria/principle	Final evidence still required
Keyboard and focus	Skip link, logical order, visible focus, route focus, dialog trap/restore, Escape, and safe Left/Right focus navigation.	WCAG 2.1.1, 2.1.2, 2.4.3, 2.4.7; ISO controllability.	Complete every workflow with keyboard and record tester/browser evidence.
Responsive reflow	Shared breakpoints, wrapping controls, table/card adaptation, mobile forms, and constrained dialogs.	WCAG 1.4.10 and 1.4.12.	Retest 320, 375, 768, 1024, 1440, landscape, text spacing, and native 200% zoom.
Forms and errors	Persistent labels, required state, aria-invalid, linked errors, first-invalid focus, and recovery instructions.	WCAG 3.3.1–3.3.3 and ISO error tolerance.	Manual announcement and recovery checks with NVDA.
Semantics	Headings, landmarks, named controls, tables/regions, tabs, dialog semantics, and alternative text.	WCAG 1.1.1, 1.3.1, 2.4.6, 4.1.2.	Verify reading order and meaningful names in rendered states.

Status feedback	Polite success status, assertive errors, atomic notices, and focus after consequential actions.	WCAG 4.1.3 and ISO self-descriptiveness.	NVDA checks for AI, applications, verification, privacy, and administration.
Visual presentation	Strengthened contrast, non-color status text, resizable layout, and reduced-motion behavior.	WCAG 1.4.3, 1.4.10; reduced motion advisory.	Post-remediation axe scans and manual visual checks.
User control	Manual alternatives to AI/voice, editable transcripts, confirmation, cancel/close behavior, and clear state feedback.	ISO 9241-110 controllability, conformity, individualization, and learning.	Representative PwD usability sessions.

13.2 Keyboard Acceptance Plan

Every user-facing workflow must be completable without mouse, touch, or voice. The first Tab exposes Skip to main content; focus follows reading order and remains visible; route changes focus the main region; Enter and Space activate controls; Escape closes dismissible overlays; and dialogs trap focus and restore it to their trigger. Tab and Shift+Tab retain their standard sequence.

The requested Right Arrow and Left Arrow navigation moves to the next or previous ordinary focusable control only when native behavior is not overridden. Text inputs and textareas keep caret movement until the relevant boundary. Selects, radio groups, sliders, date/time fields, editable content, media controls, and composite widgets retain their native arrow semantics. Within a dialog, focus navigation cannot leave the dialog.

13.3 ISO 9241-110 Interaction Principles

Suitability for the task is supported by role-specific dashboards and direct workflows. Self-descriptiveness is supported by labels, instructions, statuses, and explanations. Controllability is supported by manual alternatives, editable AI output, cancel/close behavior, and user-paced navigation. Conformity with expectations is supported by consistent controls and standard keyboard conventions. Error tolerance is supported by linked validation, focused recovery, confirmation, and non-destructive failure handling. Individualization is supported by text, keyboard, mouse, and

optional voice modes with language choice and reduced motion. Suitability for learning is supported by Voice Help, clear commands, feedback, and predictable navigation.

13.4 Conformance Boundary

The report does not claim complete WCAG 2.1 Level AA conformance. The latest automated matrix completed 104 route-scenario scans with no axe findings, horizontal overflow, console errors, failed requests, or scan errors. This evidence is complemented by the reported Arc en Ciel/JAWS session, but the field protocol and outcomes must be fully documented and failed cases retested before a conformance decision.

14 Appendix F — Outcome Logging, Accessible Notifications, Testing, and Performance

14.1 Outcome and Notification Data

Application submission and employer status changes create chronological outcome events containing application and actor identifiers, previous/new status, mail-delivery result, and timestamp. Events exclude documents, filenames, disability information, profile narrative, reviewer notes, passwords, and email bodies. Repeating an unchanged status must not create a duplicate outcome or email.

Employer transitions to pending, in review, accepted, or rejected trigger a plain-text candidate email when mail is configured. Mail failure never rolls back the saved business decision; sent or failed is recorded and returned. Verifier approval/rejection actions are also audited. Candidate, employer, verifier, and administrator feedback uses the shared Accessible Notice: errors are assertive alerts, success is a polite status, both are atomic, and focus may move to the outcome after an action.

14.2 Recorded Automated Verification

Table 9: Recorded automated verification results

Verification suite	Recorded result	Scope	Interpretation
API integration	25/25 passed	Public catalogues, valid role sessions, representative reads, anonymous rejection, and wrong-role rejection.	Read-only local integration baseline.
Browser E2E	26/26 passed	Public routes, protected redirects, four dashboards, wrong-role redirect, validation focus, alert, and skip link.	Automated browser coverage; does not replace manual assistive-technology testing.

Voice navigation	52/52 passed	Registry boundaries, roles, actions, clarification, dynamic records, and safety behavior.	Deterministic boundary tests; real accent and microphone evaluation remains pending.
Scoring engine	17/17 passed	Feasibility, assistance, mandatory gate, missing assessment, multi-disability, education, and weighting.	Validates the mathematical policy, not real-world placement effectiveness.
Frontend quality	Build and ESLint passed	Production bundle creation and static source checks.	Browser and deployment behavior still require runtime tests.
Performance	Zero budget violations	API latency, page load/FCP, transfer size, resource timing, and gzip bundle size.	Development-machine baseline; concurrency and deployed-origin measurements remain pending.

14.3 Performance Measurements

The 4 September local p95 API values were 358.5 ms for public jobs, 144.0 ms for job definitions, 217.8 ms for candidate session, and 132.3 ms for candidate profile. Page load/FCP p95 values were 56.5/168 ms for Welcome, 67.9/104 ms for Sign In, and 48.3/84 ms for Candidate. The largest JavaScript and CSS assets were 150,516 and 6,050 gzip bytes. All configured budgets passed with zero violations.

The first performance run identified a synchronous external Google Fonts request capable of delaying rendering by approximately 6.2 seconds. The dependency was removed and system font fallbacks were used, after which repeated production-build measurements passed all budgets. These measurements are a regression baseline on the development computer—not evidence of production availability, 100 concurrent users, mobile hardware performance, or external-network behavior.

14.4 Required Manual and Deployment Retests

Final manual evidence must document the Arc en Ciel/JAWS session, keyboard completion, native zoom, dialog focus, AI suggestion review, privacy operations, mobile reflow, and participant feedback. Staging evidence must be expanded with repeated deployed-origin performance, multi-device/network checks, bounded concurrency, monitoring, backup/erasure, purge scheduling, email delivery, and incident-response procedures.

15 Appendix G — Documentation and Evidence Traceability

The report contains the engineering content needed to understand the solution without opening GitHub. Repository links remain useful for verifying exact source code, commands, historical decisions, and machine-readable evidence; they are not substitutes for the written documentation.

Table 10: Repository documentation incorporated into this report

Repository document	Subject incorporated in report	Report location	Treatment
AI_PROFILE_WORKFLOW.md	Candidate journey, consent, validation, privacy boundaries, scoring invariant, and endpoints.	Appendix C	Written into the report; link retained only for source traceability.
AUTHORIZATION_SECURITY.md	JWT model, server-confirmed frontend route protection, role boundaries, and regression audit.	Appendix B	Written into the report.
CANDIDATE_VERIFICATION.md	Two-gate registration, verifier workflow, protected documents, audit, retention, and deployment.	Appendix B	Written into the report.
PRIVACY_CONTROLS.md	Consent, minimization, private storage, export, withdrawal, erasure, and operational limits.	Appendix B	Written into the report.
scoring-engine-technical-specification.md	Formula, gates, policy constants, ranking, data assumptions, determinism, tests, and limitations.	Appendix D	Core technical and mathematical content written into the report.
voice-navigation-technical-specification.md	Architecture, command safety boundary, languages, lifecycle, actions, privacy, configuration, and testing.	Appendix C	Core technical content written into the report.
frontend-revamp.md	Architecture evolution, candidate/employer/admin workflows, catalogue import, voice actions, and delivery history.	Main progress section and Appendices A–C	Consolidated by current feature rather than copying obsolete chronological notes.
keyboard-navigation-test-plan.md	Acceptance criteria, workflow matrix, dialogs, screen-reader checks, and arrow-key boundaries.	Appendix E	Written into the report with pending manual evidence identified.
wcag-audit/*.md	Baseline scope, responsive foundation, route reviews, management reviews,	Appendix E	Consolidated into an evidence and retest matrix;

	shared corrections, issue matrix, and final audit status.		no unsupported conformance claim.
OUTCOME_LOGGING_AND_ACCESSIBLE_NOTIFICATIONS.md	Outcome-event minimization, mail status, shared accessible notices, and manual retest.	Appendix F	Written into the report.
INTEGRATION_E2E_PERFORMANCE.md	Executable suites, results, performance values, and limitations.	Appendix F	Written into the report and illustrated by Figure 12.
project-setup-and-docker-guide.md	Architecture, prerequisites, setup, persistence, database access, updates, and troubleshooting.	Appendix A	Operational essentials written into the report.
run-project-commands.md	Startup, shutdown, migrations, fixtures, tests, logs, inspection, and recovery commands.	Appendix A	Validated command set written into the report.
Machine-readable JSON evidence	WCAG scenarios, integration, E2E, and performance measurements.	Appendices E–F	Results summarized; raw JSON remains attached to source control to preserve machine readability.

Machine-generated JSON is intentionally summarized rather than printed line by line because raw output is not readable report prose. The original JSON files remain versioned evidence that can be rerun and compared. Chronological frontend notes are likewise consolidated into the current architecture and behavior so obsolete intermediate decisions are not presented as the final system.

Complete repository documentation: [docs directory on branch staging-deployment](#)

Current implementation source: [inclusive-web-platform branch staging-deployment](#)

16 Appendix H — Cloud Staging Deployment Evidence

16.1 Deployment Topology and Boundaries

The Render Blueprint deploys the React/Vite frontend as a static site and the Symfony API, deterministic scoring engine, and AI/voice service as independent Docker web services. The frontend uses only public HTTPS origins. Symfony is the only component authorized to send official scoring payloads or persist candidate-confirmed AI suggestions. Shared random tokens authenticate Symfony-to-Python traffic. Supabase supplies the SSL-enabled PostgreSQL pooler and three purpose-specific storage buckets; its secret key remains server-side.

16.2 Security and Data Controls

The static site applies nosniff, strict-origin referrer, frame-denial, and microphone permission headers. CORS permits only the deployed frontend origin. Candidate verification documents and application documents are private; employer logos are intentionally public. JWT keys are reconstructed from Base64 environment secrets during container startup, with the private key restricted to the root/web-server group. Database, Supabase, OpenAI, JWT, and service-token values are absent from Git and this report.

16.3 Data Migration and Verification

A PostgreSQL custom-format backup of the controlled local database was restored into Supabase. Aggregate checks recorded 9 user accounts, including 1 administrator, 1 verifier, 4 employers, and 3 candidates; 15 job descriptions; 923 tasks; 14,453 assessments; and the associated applications and profile records. Additional demonstration offers were inserted idempotently. The public overview showed 74 published opportunities on 8 September 2026. Authentication and representative authorized reads were then verified for all four roles.

16.4 Operational Lessons

Two deployment failures produced useful engineering evidence. Database authentication failed until the newly confirmed Supabase password was committed through the dashboard and inserted into the Render URI. Backend startup later failed because the web process could not read the generated JWT private key; ownership and permissions were corrected in the container entrypoint. Free-service cold start was also observed as a temporary 503 from candidate matching. A successful repeat after health warm-up showed that the scoring calculation and candidate data were valid. These incidents define concrete demonstration checks and future monitoring requirements.

17 Appendix I — Human Evaluation and Statistical Workbook

17.1 Participant and Session Record

[TO COMPLETE FOR EACH SESSION] Participant code; role; relevant interaction needs; consent; date; location; facilitator; observer; device; operating system; browser; assistive technology and version; language; network; prior familiarity; deviations; and evidence-file references. Do not include names, disability documents, credentials, recordings, or quotations without explicit permission.

17.2 Task-Level Dataset

For every task, record a stable task identifier, expected outcome, completion status (independent, assisted, failed, or not attempted), start/end time, error count, recovery count,

assistance type, issue severity, participant comment, observer note, defect identifier, correction commit, and retest status. Retain raw anonymized rows so all percentages and charts can be regenerated.

17.3 Planned Calculations

Task completion (%) = completed tasks / attempted tasks × 100. Independent completion (%) = independently completed tasks / attempted tasks × 100. Assistance rate (%) = assisted completed tasks / attempted tasks × 100. Error rate = observed errors / attempted tasks. Report median and interquartile range for time and errors when distributions are skewed or samples are small. Present results by role, access mode, workflow, and language, followed by an overall value only when aggregation is meaningful.

17.4 Evidence Integrity

Keep pre-correction and post-correction results separate. Never convert assisted completion into independent completion, remove failed tasks from the denominator without explanation, or describe a single participant as representative of all users. Every statistic in the final report must be reproducible from the anonymized workbook and connected to the relevant defect or conclusion.